



Article

Early Ransomware Detection with Deep Learning Models

Matan Davidian [†], Michael Kiperberg [†] and Natalia Vanetik ^{*}

Department of Software Engineering, Shamoon College of Engineering, Beer Sheva 84100, Israel; davidianmatan@gmail.com (M.D.); mkiperberg@gmail.com (M.K.)

^{*} Correspondence: natalyav@sce.ac.il

[†] These authors contributed equally to this work.

Abstract: Ransomware is a growing-in-popularity type of malware that restricts access to the victim's system or data until a ransom is paid. Traditional detection methods rely on analyzing the malware's content, but these methods are ineffective against unknown or zero-day malware. Therefore, zero-day malware detection typically involves observing the malware's behavior, specifically the sequence of application programming interface (API) calls it makes, such as reading and writing files or enumerating directories. While previous studies have used machine learning (ML) techniques to classify API call sequences, they have only considered the API call name. This paper systematically compares various subsets of API call features, different ML techniques, and context-window sizes to identify the optimal ransomware classifier. Our findings indicate that a context-window size of 7 is ideal, and the most effective ML techniques are CNN and LSTM. Additionally, augmenting the API call name with the operation result significantly enhances the classifier's precision. Performance analysis suggests that this classifier can be effectively applied in real-time scenarios.

Keywords: ransomware; deep learning; API call sequences; cybersecurity; malware detection; behavioral analysis



Citation: Davidian, M.; Kiperberg, M.; Vanetik, N. Early Ransomware Detection with Deep Learning Models. *Future Internet* **2024**, *16*, 291. <https://doi.org/10.3390/fi16080291>

Academic Editors: Raha Moraffah and Li Yang

Received: 20 July 2024

Revised: 5 August 2024

Accepted: 9 August 2024

Published: 11 August 2024



Copyright: © 2024 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

1. Introduction

Ransomware (RW) is malware that prevents access to a computer system or data until a ransom is paid. It is primarily spread via phishing emails and system flaws, and it has a serious negative impact on individuals and companies that use computer systems daily [1–3].

In general, ransomware can be divided into two main types. The first type is called locker ransomware. It aims to deny access to a computer system but does not encrypt files. This type of RW blocks users from the system interface and locks them out of their work environments and applications [4]. The second type is called crypto ransomware. It encrypts valuable data in the system, such as documents and media files, and it renders them inaccessible without a decryption key. This is the dominant form of RW because of its devastating effect on data integrity [4].

The effects of ransomware attacks go beyond the money lost as soon as the ransom is paid. Operational interruptions can cause major productivity losses for organizations, particularly in vital industries like healthcare [1,2,5]. In addition, victims may experience intense psychological effects, including feelings of anxiety and violation [2]. Ransomware is a profitable business for hackers because the costs of downtime, data loss, and system recovery frequently outweigh the ransom payment itself [6]. The rate of ransomware attacks has increased significantly; in 2017, an attack occurred somewhere in the world every 40 s, and by 2019, this frequency had escalated to every 19 s [7]. Financial losses due to ransomware attacks were \$8 billion in 2018 and over \$20 billion by 2021 [8]. Ransom demands range from a few hundred dollars for personal computers to up to a million dollars for enterprises [9], with victims facing potential losses of hundreds of millions of

dollars if they do not pay. The first reported death following a ransomware attack occurred at a German hospital in October 2020 [10].

Given the sophisticated and evolving nature of ransomware, understanding its mechanics and impacts is crucial. This includes recognizing how it can infiltrate systems, the variety of its types, and the extensive consequences of attacks. Therefore, effective detection and mitigation strategies are essential when malicious activity starts. This paper contributes to these efforts by employing deep learning techniques to detect and analyze ransomware based on system behavior and response patterns within the first few seconds of its activity.

Deep learning (DL) is an excellent tool for spotting subtle and complicated patterns in data, which is important for detecting zero-day ransomware assaults [11,12]. Once trained, deep learning models can handle enormous amounts of data at rates faster than human analysts, making them perfect for real-time threat identification. These models can also identify new and changing threats over time. But big, well-labeled datasets are necessary for efficient deep learning applications, and their preparation can be costly and time-consuming [13]. Additionally, there is a chance that models will overfit, which would hinder their ability to be generalized to fresh, untested data. Finally, training deep learning models demands substantial computational resources, which can be an obstacle for some organizations [14].

Ransomware often performs operations repeatedly, for example, file scanning and the encryption of multiple directories. This conduct implies that RW contains consistent and detectable behavioral patterns. These patterns subtly evolve with each RW variant, presenting an ideal use case for deep learning models, especially those designed for sequence analysis. Moreover, the relative ease of modifying existing ransomware toolkits allows attackers to rapidly develop new variants [15]. Deep learning's capability to learn from incremental data adjustments makes it highly effective at identifying slight deviations from known behaviors, offering a robust defense against an ever-evolving ransomware landscape.

In this paper, we present a new dataset and a method for early ransomware detection. Our contribution is three-fold. First, we have created a comprehensive dataset featuring a wide array of initial API call sequences from commonly used benign and verified crypto-ransomware processes. This dataset is unique not only in its verification process, ensuring that all included ransomware samples are 100% validated as crypto-ransomware, but also in the depth of data recorded for each API call. It includes detailed information such as the result of each call, its duration, and the parameters involved. The public release of this dataset will make it a useful tool for researchers, enabling them to make even more progress in ransomware detection and stronger protection system development. Second, we have conducted a detailed comparative analysis of various neural network configurations and dataset features. This analysis aims to determine the most effective neural network model and feature set for ransomware detection. Third, we detect ransomware processes using initial API call sequences of a process and obtain an efficient method of early ransomware detection.

We examine the following research questions (RQs):

RQ1: What API call features are essential for early ransomware detection?

RQ2: Do neural models outperform traditional machine learning (ML) models for this task?

RQ3: What representation of textual API call features yields better results?

RQ4: What number of consecutive API calls from every process is sufficient for state-of-the-art results?

RQ5: Are test times for neural models competitive and suitable for online ransomware detection?

Due to the scarcity of available datasets and code, we decided to share both in order to facilitate further research in the field. Both data and code will be publicly available when this paper is published.

2. Background

Traditionally, ransomware detection methods have relied on several key strategies. Signature-based detection is the most common method used in traditional antivirus software. It matches known malware signatures—unique strings of data or characteristics of known malware—against files. While effective against known threats, this method struggles to detect new, unknown ransomware variants [16,17]. Heuristic analysis uses algorithms to examine software or file behavior for suspicious characteristics. This method can potentially identify new ransomware based on behaviors similar to known malware, but its effectiveness depends on the sophistication of the heuristic rules [18]. Behavioral analysis monitors how programs behave and highlights odd behaviors—like quick file encryption—that could indicate ransomware. Although these tools need a baseline of typical behavior and can produce false positives, they may identify zero-day ransomware attacks (new and undiscovered threats) [18]. Sandboxing runs files or programs in a virtual environment (sandbox) to observe their behavior without risking the actual system. If malicious activities like unauthorized encryption are detected, the ransomware can be identified before it harms the real environment. However, some advanced ransomware can detect and evade sandboxes [19]. Honeyfiles (decoy folders) places decoy files or folders within a system. Monitoring these decoys for unauthorized encryption or modifications can signal a ransomware attack. While useful as an early warning system, it does not prevent ransomware from infecting genuine files [20].

Although each of these approaches has advantages, they also come with special difficulties when it comes to ransomware detection. One major obstacle is finding a balance between the necessity for quick, precise detection and the reduction in false positives and negatives. For this purpose, machine learning technologies, especially deep learning (DL), are now used because they provide strong defenses against ransomware and other sophisticated cyber threats. DL is used in malware classification [21], phishing detection [22], anomaly identification [23], and malware detection. By examining the order of operations in a system, which may include odd file-encryption activities, DL models have demonstrated high efficacy in detecting ransomware activities [24,25]. DL can spot subtle and complicated patterns in data, which is important for detecting zero-day ransomware assaults; it can also handle enormous amounts of data, making it perfect for real-time threat identification. However, big and well-labeled datasets are necessary for efficient DL models, and their preparation can be costly and time-consuming [13]. Additionally, there is a chance that models will overfit and not generalize well on fresh and untested data. Training DL models demands substantial computational resources, which can be an obstacle for some organizations [14].

Next, we survey some of the most prominent works on ML-based ransomware detection. The study [26] significantly extends the realm of cybersecurity by utilizing an advanced dataset consisting of both ransomware and benign software samples collected from 2014 to early 2021. These samples underwent dynamic analysis to document API call sequences, capturing detailed behavioral footprints. The LightGBM model was used to classify the samples. The model demonstrated exceptional efficacy, achieving an accuracy of 0.987 in classifying software types.

The work [27] presents a sophisticated approach to malware detection by distinguishing API call sequences using long short-term memory (LSTM) networks, which were not limited to ransomware. The dataset in this paper was sourced from Alibaba Cloud's cybersecurity efforts, and it contains a comprehensive collection of malware samples, including ransomware. The dataset spans various malware types, and it includes dynamic API call sequences from malware, capturing only the names of the API calls while omitting additional details such as call results or timestamps. API call sequences are mapped from strings into vectors using an API2Vec vectorization method based on Word2Vec [28]. The LSTM-based model of [27] achieved an F1-score of 0.9402 on the test set, and it was shown to be notably superior to traditional machine learning models.

The paper [29] introduces an innovative approach to malware detection using deep graph convolutional neural networks (DGCNNs) [30]. It focuses on the capabilities of DGCNNs to process and analyze API call sequences. The dataset used in this work comprises 42,797 malware API call sequences and 1079 goodware API call sequences; only the API call names were recorded. DGCNNs demonstrated comparable accuracy and predictive capabilities to LSTMs, achieving slightly higher F1 scores on the balanced dataset but performing less well on the imbalanced dataset.

The work [31] concentrates on the behavioral analysis of both malicious and benign software through API call monitoring. Instead of analyzing the sequence of API calls, this study employs advanced machine learning techniques to assess the overall frequency and type counts of these API calls. The authors developed two distinct datasets that include a wide variety of ransomware families. The datasets contain only API call names. Several ML algorithms were tested, including k-nearest neighbors (kNNs) [32], random forest (RF) [33], support vector machine (SVM) [34], and logistic regression (LR) algorithms [35]. Both LR and SVM exhibited exemplary performance, achieving perfect precision scores of 1.000 and the highest recall rates of 0.987, which correspond to an F1-score of 0.994.

In the paper [29], the authors propose a novel behavioral malware detection method based on deep graph convolutional neural networks (DGCNNs) to learn directly from API call sequences and their associated behavioral graphs. They use longer call sequences (100 API calls) to achieve high classification accuracy and F1 scores on a custom dataset of over 40,000 API call sequences.

The goal of this paper is to improve detection skills by combining deep learning with the fundamentals of behavioral analysis. This will lessen the possibility of false positives and improve the identification of zero-day threats.

3. PARSEC Dataset

3.1. Motivation

One of the primary reasons for opting to collect our data, rather than using pre-built datasets, was the lack of available datasets that include detailed outcomes of API calls. Most publicly available datasets (described in Section 2) typically provide only the names of the API calls made during the execution of malware and benign applications. We made an effort to find a dataset that would fit our research. The reviewed datasets were MalBehavD-V1 [36], the Malware API Call Dataset [37], the Alibaba Cloud Malware Detection Based on Behaviors Dataset [38], and the datasets introduced in papers [29,39]. None of these datasets were suitable for our purposes because they only provided the names of API calls. In our study, we wanted to explore the effect of additional information, such as the result of the API call, its duration, and the parameters it received, to see whether these additional details could improve performance metrics in ransomware detection.

However, for a more nuanced analysis and potentially more effective detection models, it is crucial to consider not only the API calls themselves but also their results, the duration of each call, and its parameters. This is why we present a new dataset named PARSEC, which stands for API calls for ransomware detection.

3.2. Data Collection

We chose to use Windows 7 for malware analysis because, despite being an older operating system, it remains a target for malware attacks due to its widespread use in slower-to-upgrade environments [40]. Therefore, malware analysis on Windows 7 provides insights into threats still exploiting older systems. Additionally, many malware variants designed to target Windows platforms maintain compatibility with Windows 7 due to its architectural similarities with newer versions. Note that our method and results are also applicable to the Windows 10 OS and its server counterparts. It is important to note that, for our purposes, Windows 7 and Windows 10 are API-compatible. This means a sample identified as ransomware on Windows 7 would also be classified as ransomware on Windows 10.

We used Process Monitor [41] (PM) on a Windows 7 Service Pack 1 (SP1) environment within VirtualBox v6.1 [42] to record API calls from both benign and malicious processes. Process Monitor (v3.70) is a sophisticated tool developed by Sysinternals (now part of Microsoft) that can capture detailed API call information [43].

We collected the data for malicious and benign processes separately. For each API call of a process, we recorded the call's name, result, parameters, and execution time. Then, we filtered the API calls and their parameters from every process to construct our datasets; this procedure is shown in detail below. Figure 1 shows the pipeline of our data collection.

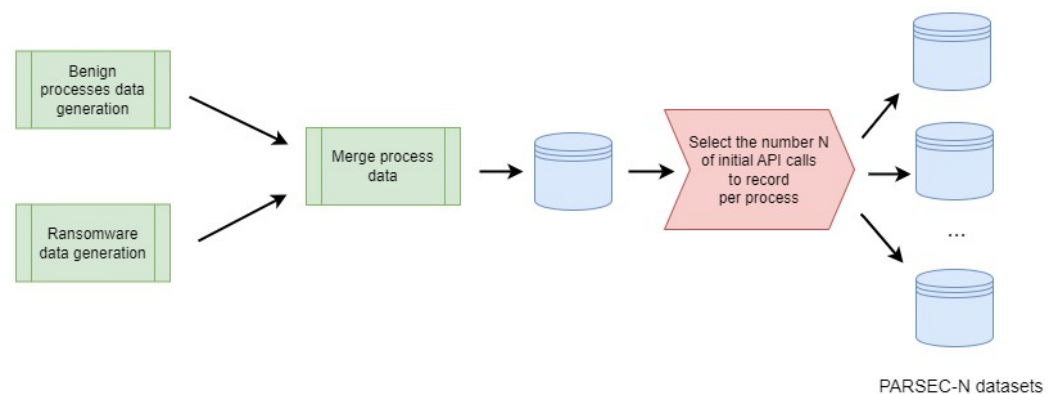


Figure 1. Data collection pipeline.

Note that our set of ransomware and benign processes is extensive, but it does not contain all possible processes. However, choosing a few representatives from each application group is an acceptable practice in benchmarking, and we consider the selected set of benign applications to be adequately representative. While ransomware can theoretically differ significantly, in practice, it generally follows the same patterns as other malware. There are several notable examples, such as Locky, WannaCry, and Ryuk, from which all others are derived [44,45].

3.3. Benign Processes

We selected a diverse suite of 62 benign processes to capture a broad spectrum of normal computer activities. This selection strategy was aimed at ensuring that our dataset accurately reflects the varied operational behaviors a system might exhibit under different scenarios, including active user interactions and passive background tasks. These processes belong to five main types described below.

- Common applications, such as 7zip (v22.01), axCrypt (v2.1.16.36), and CobianSoft (v2.3.10), are renowned for their encryption and backup capabilities. These choices are important for studying legitimate encryption activities, as opposed to the malicious encryptions conducted through ransomware.
- Utility and multimedia tools, such as curl (for downloading tasks) and ffmpeg (v.3, for multimedia processing), are crucial for representing standard, non-malicious API call patterns that occur during routine operations.
- Office applications like Excel (office professional plus 2010) and Word (office professional plus 2010) reflect common document-handling activities—normal document access and modification patterns.
- Benchmarking applications such as Passmark (v9) and PCMark7 (v1.4.0) simulate a wide array of system activities, from user engagement to system performance tests. These applications provide a backdrop of benign system-stress scenarios.
- Idle-state processes that typically run during the computer's idle state represent the system's behavior when it is not actively engaged in user-directed tasks. This category is essential for offering insights into the system's baseline activities.

The full list of benign processes appears in Appendix A.1 of Appendix A.

3.4. Ransomware Processes

We started from a dataset comprising 38,152 ransomware samples, obtained from [VirusShare.com](#) [46]. To verify this site's virus classification, we employed an automated pipeline to verify the authenticity of these samples as ransomware. The objective was to identify at least 62 ransomware programs within this dataset to match the number of benign processes described in Appendix A.1. The identification pipeline is a multi-stage process designed to differentiate actual ransomware from potential threats. It includes two VirtualBox virtual machines (VMs) and a host machine, each playing a critical role in screening, analyzing behavior, and confirming ransomware candidates. The full pipeline of ransomware API calls collection flow is shown in Figure 2.

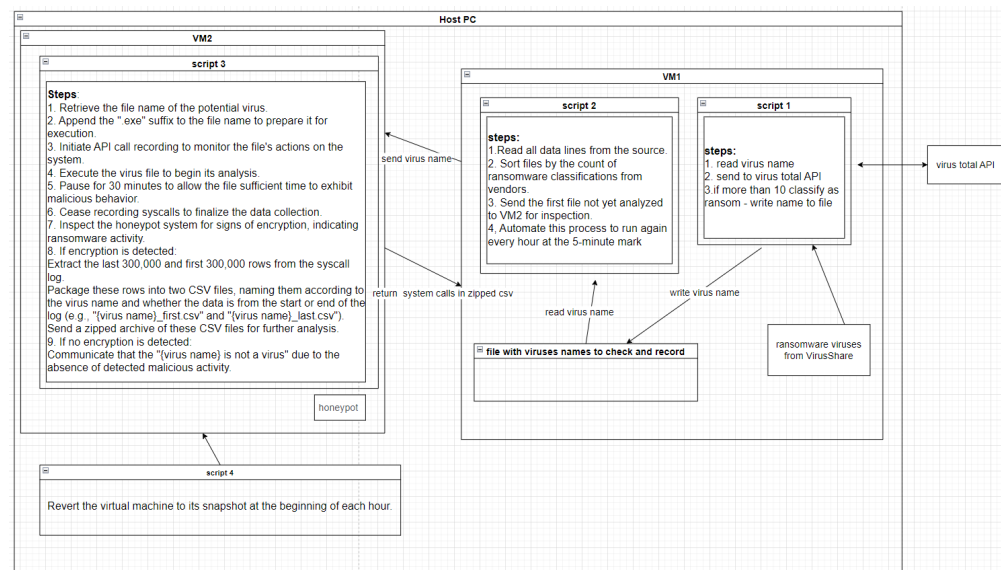


Figure 2. Ransomware verification pipeline.

The first virtual machine (denoted as VM1) starts the process by querying the VirusTotal API for each entry in the “VirusShare_CryptoRansom_20160715” collection, which consists of 38,152 potential samples. Its objective is to filter and prioritize samples based on the frequency of detections via various antivirus engines. Prioritized samples are forwarded to the second virtual machine (denoted as VM2) for a detailed behavioral analysis.

VM2 receives prioritized samples (one by one) from VM1 and executes each in a secure, controlled setting. It focuses on detecting encryption attempts targeting a “honey spot”, which refers to a deliberately crafted and strategically placed element within a system or network designed to attract ransomware or malicious activities [47]. All API calls made during execution are recorded. If a sample is confirmed as ransomware (i.e., it encrypts the “honey spot”), VM2 compresses the API call data into an Excel file, packages it with WinRAR, and sends it back to VM1.

The host machine maintains a consistent testing environment by resetting VM2 after each analysis. It gathers the compressed Excel files containing API call data from confirmed ransomware samples and compiles them into a single list of these verified programs. This process resulted in a dataset of 62 validated ransomware programs from the initial 38,152 candidates after it ran for two weeks.

3.5. Dataset Features

From the collected API calls of the PARSEC dataset, we generated several datasets that differ in the number of API calls taken from each process. We selected N initial API calls of processes to enable our models to detect malicious processes upon their startup; here, N is a parameter. The aim of our approach is the early detection of ransomware processes. If a process executes fewer API calls than required for the dataset, we performed

data augmentation using oversampling. Specifically, we replicated sequences of API calls at random; this method guarantees datasets' consistency.

We selected a number of API calls between 500 and 5000 to evaluate the potential for early ransomware detection based on limited API calls. It also helped us understand the implications of dataset size on the efficiency of our models. Note that the dataset size primarily affects the duration of training. Larger volumes of data extend the training time but may result in models that are better at generalizing across different ransomware behaviors. Conversely, smaller datasets reduce the training time but might limit the model's comprehensiveness in learning varied ransomware patterns. This balance is crucial for developing practical, deployable models that can be updated and retrained as new ransomware threats emerge. The naming convention for dataset variations is PARSEC-N, where N is the number of initial API calls included for each process. Therefore, we have PARSEC-N datasets for $N = 500, 1000, 2000, 3000, 4000, 5000$.

The API features we recorded include process operation, process result, process duration, and process detail features (a full list of these features appears in Appendix A.2). We denote these feature lists as Ops, Res, Dur, and Det, meaning operations, results, duration, and detail features. In the basic setup, we started with operation features and only extended the list by adding the result features, and then we added the API execution times and detail features. By starting with basic features and incrementally adding complexity, we isolated the impact of each feature type on the models' performance. We denote as FLIST the list of features used in the dataset; it accepts the values Ops (process operation features), OpsRes (process operation and result features), OpsResDur (process operation, result, and duration features), and OpsResDurDet (process operation, result, duration, and detail features).

3.6. Data Representation

API call names, results, and execution times were directly extracted from the raw data without modification. Process details' features are long strings representing the parameters passed to each API call in a semi-structured format. Each parameter is delimited with a semicolon (";"), with key-value pairs within these parameters separated by a colon (":"). The value of each key varied, ranging from numbers to single words or even phrases. To accurately interpret and utilize this information, we implemented a detailed extraction process:

1. First, we separated and extracted each parameter and its corresponding key-value pairs.
2. Then, we filtered out identifiable information—parameters that could serve as identifiers or indicate specific timestamps were meticulously removed to maintain the integrity of the dataset and ensure privacy compliance. The full list of these parameters can be found in Appendix A.2 of Appendix A.
3. We filled in the missing data with sequences of zeros.
4. Due to the heterogeneous nature of API calls, they might be associated with a set of parameters of different sizes. Therefore, API calls with missing parameters were systematically padded with zeros.

After feature extraction, we normalized the numerical features (such as execution times) using min-max normalization.

We used 1-hot encoding, FastText [48], and Bidirectional Encoder Representations from Transformers (BERT) sentence embeddings [49] (BERT SE) to represent text features. For FastText representation, we split all string attributes into separate words, according to camel case patterns, punctuation, tabs, and spaces, as in "END OF FILE". The text was kept in its original case. Then, we extracted the k -dimensional word vector of every word and computed its average vector. We used fastText vectors pre-trained on English webcrawl and Wikipedia of length $k = 300$. For BERT SE representation, the words were split based on camel cases and spaces, and then all strings representing words were transformed into lowercase. Then, we applied a pre-trained model bert-base-uncased and extracted vectors of length 768 for every text.

Next, we divided the data into fixed-size windows of size W . We explored four window sizes, with $W = 1, 3, 5, 7$. To maintain consistency across the dataset and ensure integrity in the windowed structure, we applied zero-padding where necessary. This is particularly important for the final segments of data sequences, which may not be fully populated due to variability in API call frequencies. The full data representation pipeline is depicted in Figure 3.

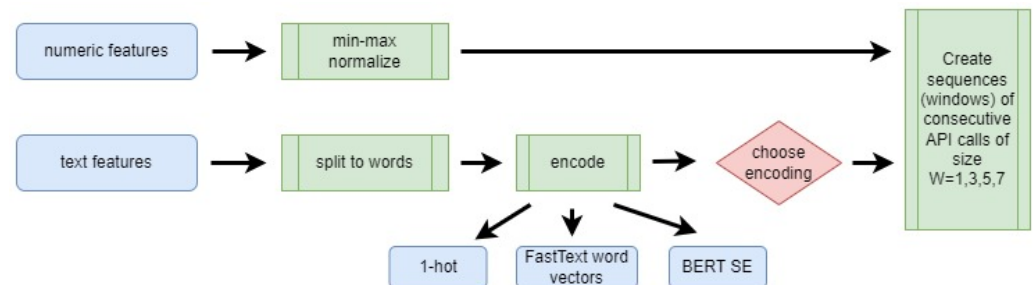


Figure 3. Data representation pipeline.

3.7. Data Analysis

We performed a visual and numeric analysis of our datasets to assess the quality and behavior of benign and ransomware processes. We focused on two datasets—PARSEC-500 and PARSEC-5000—that represent the smallest and biggest numbers of initial API calls taken from each process.

Table 1 contains the number of API calls performed by benign and ransomware processes for the PARSEC-500 and PARSEC-5000 datasets. We omitted the calls that were never performed through ransomware processes from this table (the full list of these calls is provided in Appendix A.3 of Appendix A). We can see, surprisingly, that the same calls appear when the first 500 API calls are taken (PARSEC-500) or the first 5000 (PARSEC-5000). It is also evident, in total, ransomware processes perform much more CloseFile, CreateFile, and IRP_MJ_CLOSE than benign processes do. They, however, perform fewer ReadFile operations than benign processes, regardless of the number of system calls recorded.

Table 1. Number of system calls comparison

Operation	PARSEC-500		PARSEC-5000	
	Benign	Ransomware	Benign	Ransomware
CloseFile	1168	5052	9151	22323
CreateFile	1921	5061	11044	23374
IRP_MJ_CLOSE	915	3717	6975	17031
Process Profiling	6402	228	8996	388
QueryAttributeTagFile	514	1269	1189	4645
QueryBasicInformationFile	327	3306	2210	15182
QueryDirectory	153	601	1916	4546
QueryOpen	388	1075	3636	6987
QueryStandardInformationFile	678	1230	2443	4635
ReadFile	1618	2888	19005	10593
SetBasicInformationFile	780	1123	1496	4992
SetRenameInformationFile	0	1259	3	4643
WriteFile	216	2606	5251	9986

Next, we performed a visual analysis to reveal distinguishing malware characteristics. For each process, we generated a square image where each pixel represents an API call, color-coded according to the operation performed. The images were plotted with legends, associating each color with its respective API call operation. The visual analysis revealed a stark contrast between benign and ransomware processes. Benign processes exhibited a diverse array of patterns, reflecting the wide-ranging legitimate functionalities and interactions

within the system. Each benign process presents a unique color distribution, illustrating the variability and complexity of non-malicious software operations. An example is shown in Figure 4. Visualization of other benign processes appears in Appendix A.4 of Appendix A.

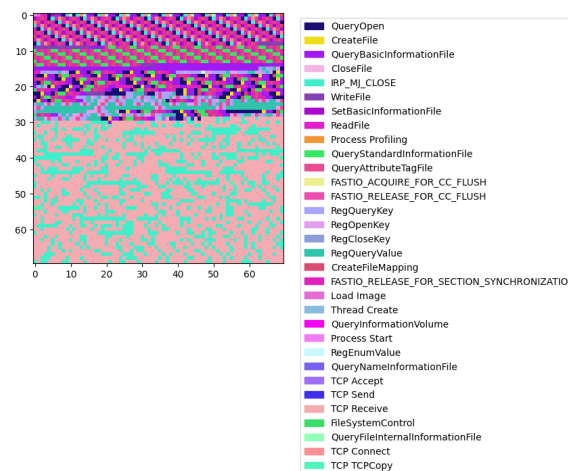


Figure 4. curl.exe v7.71.1 (downloading software).

In contrast, ransomware processes displayed a more homogenous appearance, with similar color distributions among them. This uniformity suggests a narrower set of operations being executed, which could be indicative of the focused, malicious intent of these processes. Remarkably, the ransomware processes can be grouped into a few distinct types based on the visualization of their operational sequences, suggesting the existence of common strategies employed across different malware samples.

The first type of malware (Figure 5) prominently features operations like QueryBasicInformationFile, ReadFile, and CreateFile in repetitive patterns.

The second type of malware (Figure 6) exhibits a more randomized and chaotic distribution of API calls across the images.

Finally, the third type of malware (Figure 7) displays a distinct two-part division, possibly indicating a shift from the initial setup or reconnaissance to intense malicious activity, such as data manipulation or encryption.

In total, we observed patterns unique to malicious activities visually, which implies that sequence analysis is useful for malware detection.

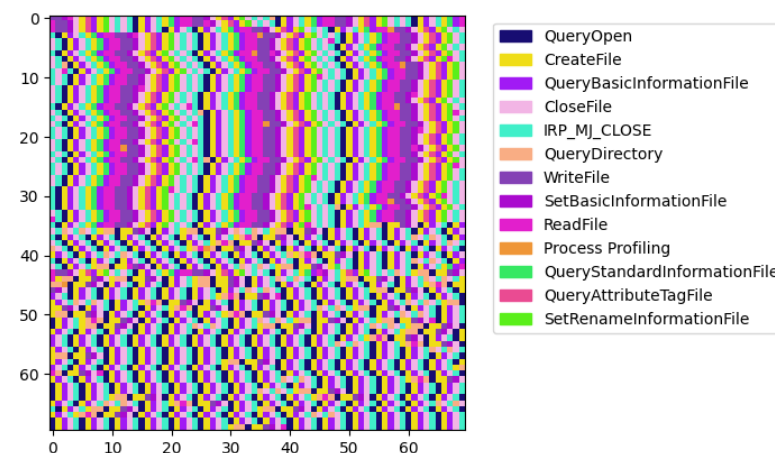


Figure 5. 0a85ea7926dbb0ea07c702d6894ca1d0.exe (ransomware sample).

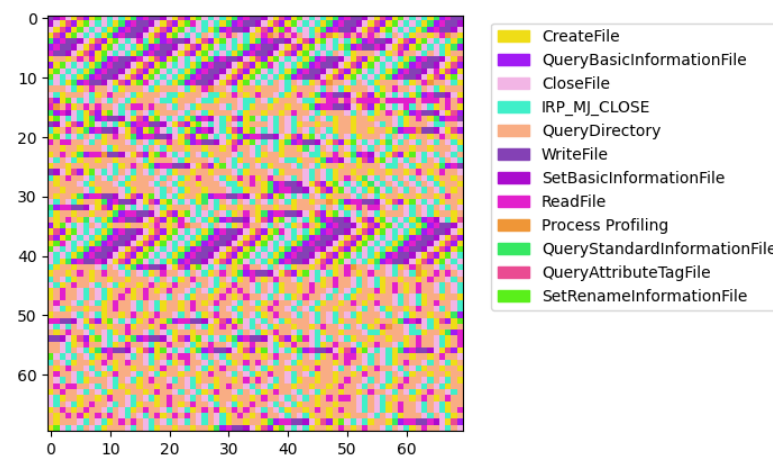


Figure 6. 0adf953605c610880f4095b3b33ea2d9.exe (ransomware sample).

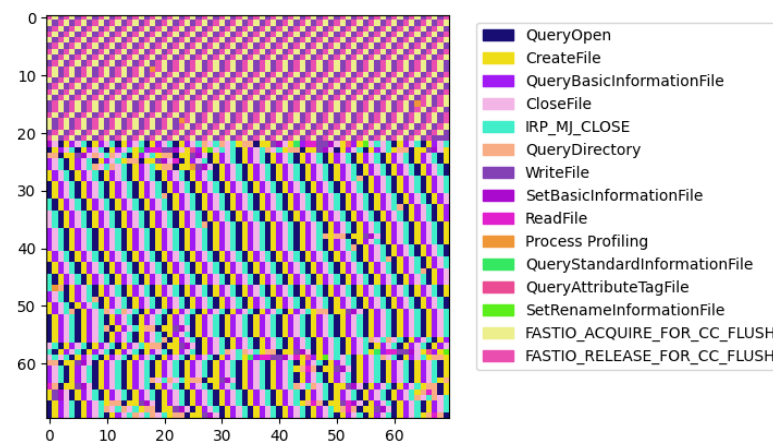


Figure 7. 7a2a1fdc535f9b9a76443231e3f8b0c4.exe (ransomware sample).

4. Method

4.1. Pipeline

To perform early ransomware detection, we first defined the list of features and the number of initial API calls for every process and selected the dataset and its features, as described in Section 3.5. At this stage, we selected data representation for text features and normalized the numeric features as described in Section 3.6. Next, we divided the data into training and test sets (see Section 4.2 below for details). Then, we selected the window size, W , and generated sequences of API calls for the training and test sets separately. Finally, we selected a machine learning model and trained and tested it on these sets (the models are described below in Section 4.3). This pipeline is depicted in Figure 8.

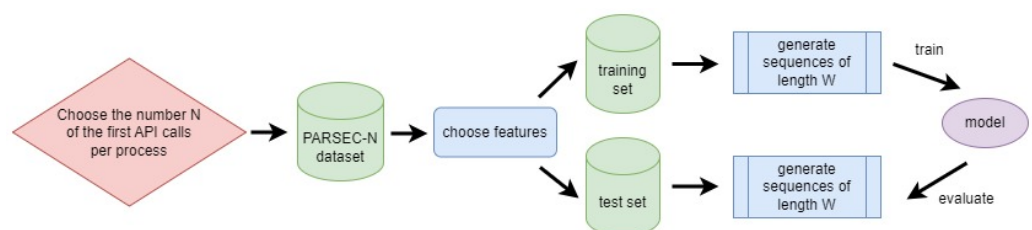


Figure 8. Ransomware detection pipeline.

4.2. Data Setup

Our dataset consists of an equal number of benign and ransomware processes, with 62 instances in each category. To form the training set, we first randomly selected 80% of

the benign processes (49 out of 62). Then, we sorted the ransomware processes based on their emergence date and included the oldest 80% (49 out of 62) in the training set. This method encourages the model to learn from historical ransomware patterns and behaviors. The remaining 20% of the benign processes (13 out of 62) were assigned to the testing set, and so were the latest 20% of the ransomware processes (13 out of 62). This aimed to assess the model's ability to detect new ransomware variants. We implemented a cross-validation strategy to further test our model's robustness against the variability in benign behaviors by creating five distinct train–test splits. In each split, while maintaining a consistent distribution of ransomware processes, we varied the benign processes included in the test set by randomly selecting a new set of 13 benign processes.

4.3. Models

We used the following neural models in our evaluation:

1. Feed-forward fully-connected neural network (DNN) with three layers (64 neurons, 32 neurons, and 1 neuron). The inner layers use ReLU activation [50], and the output layer uses sigmoid activation [51] suitable for binary classification.
2. Convolutional neural network (CNN) [52] with one convolutional layer of 32 filters, followed by a 32-unit dense layer and an output layer containing 1 neuron with sigmoid activation.
3. Long short-term memory (LSTM) [53] network with one LSTM layer with 32 neurons, followed by a 32-unit dense layer and an output layer containing 1 neuron with sigmoid activation.

All models were trained for 10 epochs, with a batch size of 16.

These neural models are easier to understand and less prone to overfitting. They are also more computationally efficient and essential for real-time detection and deployment in resource-constrained contexts. These models are probably enough because the patterns in the API call sequences we classify are not very complicated, as shown in Section 3.7.

5. Experimental Evaluation

5.1. Hardware and Software Setup

We used a desktop computer with an Intel (R) Core (TM) i7-4770 CPU @ 3.40 GHz manufactured by Intel Corporation, Santa Clara, California, United States. The desktop has 32 GB of random-access memory (RAM), 450 GB of virtual memory (with a solid-state drive (SSD) used as additional virtual memory), and an NVIDIA GeForce GTX 1050 Ti graphics processing unit (GPU) with 4 GB of graphics double data rate type five (GDDR5) manufactured by NVIDIA Corporation, Santa Clara, California, United States.

All models and tests were implemented in Python 3.8.5 and run on the Microsoft Windows 10 Enterprise Edition Operating System (OS) [54]. We used GPU graphics driver version 536.23 and CUDA version 12.2. We used the Tensorflow and Keras Python packages [55], as well as the scikit-learn [56], scipy [57], and matplotlib [58] libraries.

5.2. Metrics

These indicators represent different outcomes for binary-classification model predictions:

- TPs (true positives) are the correct predictions of the positive class (ransomware).
- TNs (true negatives) are the correct predictions for the negative class (benign processes).
- FPs (false positives) are the incorrect predictions for the negative class (ransomware).
- FNs (false negatives) are the incorrect predictions for the positive class (benign processes).

Accuracy is the overall proportion of correctly classified instances [59]:

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}}$$

We utilized the following metrics in our evaluation. Sensitivity (or recall) assessed the models' ability to correctly identify positive predictions (actual ransomware activi-

ties). In contrast, specificity measured their effectiveness in correctly classifying negative predictions (non-ransomware activities) [60]:

$$\text{Sensitivity} = \frac{TP}{TP + FN}$$

$$\text{Specificity} = \frac{TN}{TN + FP}$$

Precision measured the accuracy of positive predictions:

$$\text{Precision} = \frac{TP}{TP + FP}$$

F1 score combines precision and sensitivity into a single metric [61], offering a balanced measure of model performance:

$$\text{F1 Score} = 2 \times \frac{\text{Precision} \times \text{Sensitivity}}{\text{Precision} + \text{Sensitivity}}$$

We also measured execution times (in seconds) to get a better understanding of the models' performance:

- Test time measured the time it took for the models to evaluate all samples in the test set, and
- training time measured the duration required for the models to complete training on the entire training dataset, allowing us to assess the computational resources needed for model training.

5.3. Baselines and Models

We applied the following baseline classifiers (implemented in the scikit-learn SW package [56]):

1. Random forest (RF)—an ensemble method that uses multiple decision trees to handle large datasets effectively [33].
2. Support vector machine (SVM)—a supervised learning model effective in high-dimensional spaces but computationally intensive with large datasets [34].
3. Multilayer perceptron (MLP)—a feedforward neural network with default settings.

We used deep neural models denoted as DNN, CNN, and LSTM, described in Section 4.3, and compared our approach to the existing methods of the paper [29].

5.4. Evaluation Setup

We selected two datasets for evaluation—PARSEC-500 and PARSEC-5000. These datasets represent the two sides of the spectrum and represent the smallest and largest numbers of initial API calls recorded for every process. We evaluated the four sets of dataset features for API call representation denoted as Ops, OpsRes, OpsResDur, and OpsResDurDet (described in detail in Section 3.5). For neural models, we evaluated three options of text representations—1-hot, FastText word vectors, and BERT sentence embeddings. The training was performed for sequences of API calls of length W , and the options we tested for W were 1, 3, 5, 7. Finally, we trained our neural models for different numbers of epochs—10, 20, and 30. This setup yields 144 different configurations, with which we tested three neural models. Due to the number of results, we report the scores of the top three configurations for every dataset and then demonstrate how these models are affected by configuration changes.

5.5. Results

5.5.1. Baselines

Baseline results for traditional models (RF and SVM) on PARSEC-500 and PARSEC-5000 datasets appear in Table 2. For these models, we used 1-hot encoding of text features and window size $W = 1$, and the Ops list of data features. The RF model showcases remarkable efficiency with low test times on PARSEC-5000 dataset, indicating its scalability. It also provides much better scores on this dataset than SVM. The SVM model, despite the adjustment of the maximal iterations number that we performed, incurs significantly higher test times. This result implies that SVM is not suitable for early detection, because the system must respond quickly to a threat.

Table 2. SVM and RF F1 scores on PARSEC-500 and PARSEC-5000 datasets (the best scores are marked in gray).

PARSEC-500							
model	text repr	FLIST	sensitivity	specificity	F1	test time (s)	W
RF	1-hot	Op	0.9930	0.6484	0.8518	2.07	1
SVM	1-hot	Op	0.9930	0.6484	0.8518	42.49	1
RF	1-hot	OpRes	0.9897	0.6826	0.8624	2.85	1
SVM	1-hot	OpRes	0.9897	0.6826	0.8624	48.30	1
PARSEC-5000							
model	text repr	FLIST	sensitivity	specificity	F1	test time (s)	W
RF	1-hot	Op	0.9208	0.6658	0.8119	0.79	1
SVM	1-hot	Op	0.4128	0.8197	0.5160	444.72	1
RF	1-hot	OpRes	0.9144	0.6719	0.8108	0.92	1
SVM	1-hot	OpRes	0.4064	0.8256	0.5119	532.08	1

We evaluated our main baseline, MLP, on all feature sets, all text representations, and all window sizes ($W = 1, 3, 5, 7$). Table 3 contains the best results for all feature sets on the PARSEC-500 and PARSEC-5000 datasets (full results are shown in Appendix A.5 of the Appendix).

Table 3. MLP scores on PARSEC-500 and PARSEC-5000 datasets (the best scores are marked in gray).

PARSEC-500							
model	text repr	FLIST	sensitivity	specificity	F1	test time (s)	W
MLP	BERT SE	Op	0.9935	0.9664	0.9802	0.03	7
MLP	BERT SE	OpRes	0.9967	0.9372	0.9679	0.13	7
MLP	BERT SE	OpResDur	0.9805	0.9372	0.9597	0.09	7
MLP	BERT SE	OpResDurDet	0.9827	0.9317	0.9583	0.47	7
PARSEC-5000							
model	text repr	FLIST	sensitivity	specificity	F1	test time (s)	W
MLP	1-hot	Op	0.9989	0.9801	0.9896	0.04	7
MLP	1-hot	OpRes	0.9999	0.9849	0.9925	0.05	7
MLP	FastText	OpResDur	0.9583	0.9490	0.9539	0.38	7

MLP achieved higher scores than traditional baselines on all datasets, showing that neural networks are more suitable for our domain. We observed, however, that adding more API call features did not necessarily improve the results. The MLP model exhibited much lower test times compared to the RF model in all cases, indicating that it is more suitable for the task of early detection. The results also reveal that the best results were achieved for $W = 7$. However, not all feature sets and text representations (such as OpResDurDet) were feasible for training the MLP model in a reasonable time.

5.5.2. Top Neural Models

Table 4 shows the top three neural model setups that achieved the best F1 scores for the PARSEC-500 and PARSEC-5000 datasets.

Table 4. Top-performing models for PARSEC-500 and PARSEC-5000 datasets (the best scores are marked in gray, ↓ and – indicate the statistical significance of differences in the results, or the lack thereof).

Top 3 models for the PARSEC-500 dataset								
model	text repr	FLIST	sensitivity	specificity	F1	W	ep	test time (s)
CNN	1-hot	OpsRes	0.9978	0.9827	0.9903	7	10	0.20
CNN	1-hot	OpsRes	0.9957	0.9805	0.9882–	7	30	0.33
LSTM	1-hot	OpsRes	1.000	0.9705	0.9877↓	7	10	0.53
Top 3 models for the PARSEC-5000 dataset								
model	text repr	FLIST	sensitivity	specificity	F1	W	ep	test time (s)
LSTM	1-hot	Ops	0.9998	0.9885	0.9942	7	10	1.42
DNN	1-hot	OpsRes	0.9997	0.9870	0.9934–	7	10	0.59
CNN	1-hot	OpsRes	0.9972	0.9890	0.9931↓	7	10	0.79

The best performances were consistently observed with a window size of 7. The combination of operation and result features consistently led to the highest performance metrics. The 1-hot encoding of textual features proved to be the most effective method, outperforming other encodings in nearly all scenarios. Among the models, CNN was the standout model for API call volumes of 500. For the largest dataset of 5000 API calls, LSTM with only the operation feature performed the best in terms of accuracy, but it was slower compared to the other models. This points to a trade-off between performance and efficiency, with LSTM improving accuracy at the cost of speed.

We applied a pairwise two-tailed statistical significance test [62] to predictions of the top three models for each dataset. On PARSEC-500, the test showed that the difference between model 1 and model 2 was not statistically significant, while the difference between model 1 and model 3 was significant. Similarly, on PARSEC-5000, the test showed that the difference between model 1 and model 2 was not statistically significant, while the difference between model 1 and model 3 was significant. These results appear in Table 4 next to the F1 scores as – (the difference from the model above is not significant) and ↓ (the difference from the model above is significant).

5.5.3. Competing Model

In reviewing the literature on ransomware detection, we learned that most studies do not share their code, which hinders reproducibility and comparative analysis. After examining numerous papers in this field, such as [26,31,39,63–65], we found that they provide method descriptions but not the implementation code. The only work that shares its code is [29]. Therefore, we ran the two models presented in this work on our datasets and compared them with our models.

To evaluate the effectiveness of our proposed models, we compared our results with those obtained using a previous methodology described in the paper [29]. The method of [29] utilizes windows with a length of 100. We have used the publicly available implementation of this method. We ran the two deep graph neural network (DGNN) models (denoted by DGNN1 and DGNN2) contained in this implementation.

Table 5 shows the comparison of this method and our top three models on the PARSEC-500 dataset. All our models yielded higher F1 scores, demonstrating the robustness and effectiveness of our approach. These results highlight the improvements in detection accuracy achieved by incorporating operation and result features with 1-hot encoding.

Table 5. Comparison with the models of [29] on the PARSEC-500 dataset (the best score is marked in gray).

PARSEC-500 Dataset					
Model	text repr	FLIST	W	ep	F1
CNN	1-hot	OpsRes	7	10	0.9903
CNN	1-hot	OpsRes	7	30	0.9882
LSTM	1-hot	OpsRes	7	10	0.9877
DGNN1	-	-	-	-	0.9848
DGNN1	-	-	-	-	0.9774

5.5.4. Data Preparation Times

To verify that our best models are suitable for practical online RW detection, we measured the time it took to prepare the data before they were passed on to a model for detection. We performed these tests for different feature sets and text representations. These times (per entire test set) are reported in Table 6; we separated between data normalization, windowing, and text-feature encoding. Text encoding is a more time-consuming task, and its time rises with the expansion of feature sets. However, since the best models for both datasets use feature sets Ops and OpsRes, data preparation times for these setups are feasible for practical RW detection. On the PARSEC-500 dataset, the best neural model (CNN) uses a 1-hot text representation and OpsRes feature set. This combination takes less than 1 s to prepare for the entire test set of processes. This is also the case for the best model on the PARSEC-5000 dataset (LSTM) that uses 1-hot text representation and an Ops feature set.

Table 6. Data preparation times for PARSEC-500 and PARSEC-5000 datasets (top models configurations are marked in gray).

PARSEC-500 dataset				
FLIST	text repr	normalization+ windowing (s)	text features encoding (s)	total time (s)
Ops	BERT SE	0.02	0.10	0.12
Ops	FastText	0.01	0.90	0.91
Ops	1-hot	0.02	0.02	0.04
OpsRes	BERT SE	0.01	0.14	0.15
OpsRes	FastText	0.01	1.56	1.58
OpsRes	1-hot	0.01	0.01	0.02
OpsResDur	BERT SE	1.84	0.17	2.01
OpsResDur	FastText	0.87	1.57	2.44
OpsResDur	1-hot	0.40	12.00	12.41
OpsResDurDet	BERT SE	1.96	34.06	36.02
OpsResDurDet	FastText	1.03	6.69	7.72
OpsResDurDet	1-hot	0.55	12.30	12.85
PARSEC-5000 dataset				
FLIST	text repr	normalization+ windowing (s)	text features encoding (s)	total time (s)
Ops	1-hot	0.36	0.10	0.46
Ops	BERT SE	0.40	4.80	5.20
Ops	FastText	0.36	5.24	5.60
OpsRes	1-hot	0.33	0.14	0.47
OpsRes	BERT SE	0.40	1.25	1.65
OpsRes	FastText	0.35	9.06	9.42
OpsResDur	1-hot	3.40	8066.17	8069.58
OpsResDur	BERT SE	12.86	1.14	14.00
OpsResDur	FastText	6.36	9.11	15.47

5.6. Error Analysis

In the error-analysis phase, we utilized t-Distributed Stochastic Neighbor Embedding (t-SNE) [66], a powerful algorithm for dimensionality reduction, which is well suited to the visualization of high-dimensional datasets. Our primary goal with this analysis was to identify patterns and clusters in models' predictions, specifically focusing on distinguishing between correctly classified instances and errors.

We transformed the test data into a two-dimensional space with t-SNE and plotted the two-dimensional features with plot points color-coded to distinguish between correctly classified instances (in light gray) and errors (in red). This visualization reveals areas where the model performs well and highlights regions where errors are concentrated.

In both t-SNE visualizations (shown in Figures 9 and 10), errors, represented by red dots, are interspersed among correctly classified instances, rather than clustering in isolated areas. This pattern suggests that the errors do not stem from distinct, well-defined regions of the feature space. Instead, they appear to be spread throughout, indicating that these misclassifications are not readily separable based on the model's current understanding of the features. This dispersion of errors points to the intrinsic difficulty of the classification task, where simple linear separability is not achievable, and more complex decision boundaries are necessary. Furthermore, we observe substantial regions within the t-SNE plots where correctly classified samples are dominant, with no errors nearby. This implies that, for a significant portion of the dataset, the model can classify instances with high confidence and accuracy. Such regions are indicative of samples that are likely easier to classify, either because they have more distinct feature representations or they fall far from the decision boundary within the feature space.

Overall, while the model showed competence in accurately classifying a large fraction of the data, the scattered errors highlight the challenges present in the more ambiguous regions of the feature space.

5.7. Ablation Study

5.7.1. The Effect of Text Representation

In this section, we assess the effect that textual feature representation has on the scores of the top models described in the previous section. We report the results these models achieved on the PARSEC-500 and PARSEC-5000 datasets with a window size of 7 when 1-hot vectors, FastText vectors, or BERT sentence embeddings were chosen to encode textual features (see Section 3.6 for details).

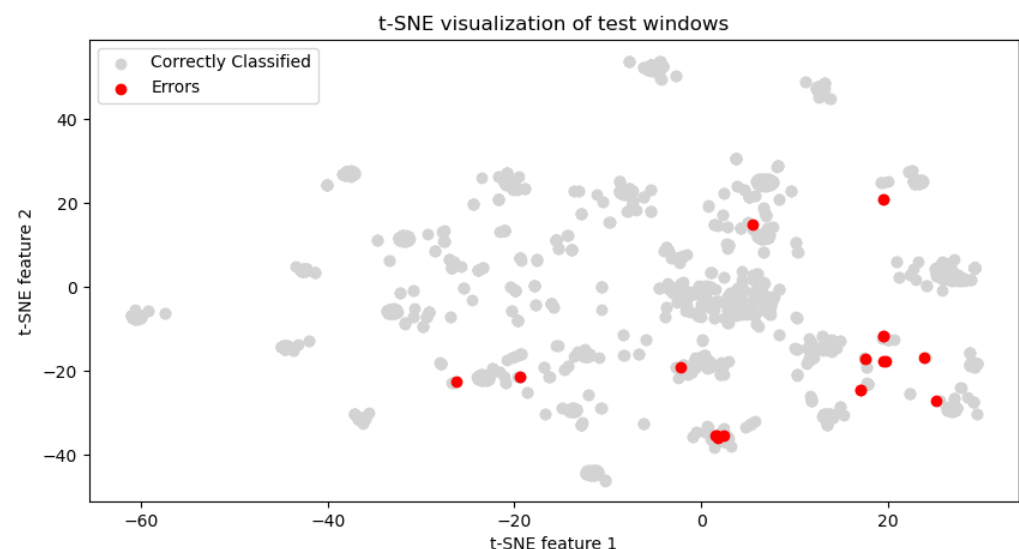


Figure 9. Error analysis—PARSEC-500 dataset with the top model (CNN 1-hot OpsRes W = 7 10 eps).

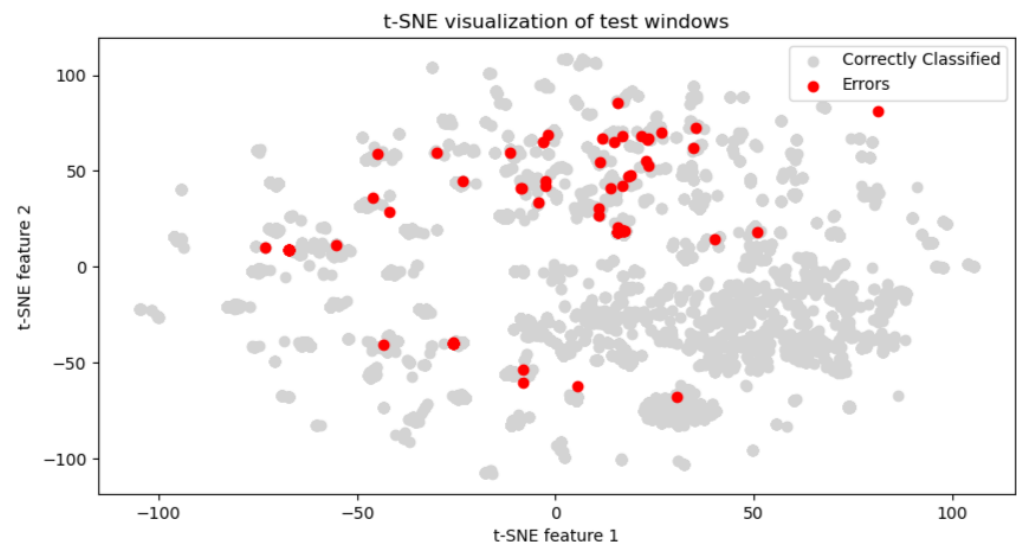


Figure 10. Error analysis—PARSEC-5000 dataset with the top model (LSTM 1-hot Ops $W = 7$ 10 eps).

F1 scores, sensitivity, specificity, and test times for tests appear in Table 7. Full results for all models, dataset features, and all window sizes are available in Appendix A in Appendices A.5 and A.6.

Table 7. F1 scores of the top models on PARSEC-500 and PARSEC-5000 datasets with different text representations (the best scores are marked in gray).

PARSEC-500 dataset								
model	text repr	FLIST	sensitivity	specificity	F1	W	ep	test time (s)
CNN	BERT SE	OpsRes	0.9989	0.9296	0.9654	7	10	0.40
CNN	FastText	OpsRes	0.9610	0.8787	0.9230	7	10	0.31
CNN	1-hot	OpsRes	0.9978	0.9827	0.9903	7	10	0.20
PARSEC-5000 dataset								
model	text repr	FLIST	sensitivity	specificity	F1	W	ep	test time (s)
LSTM	BERT SE	Op	0.9988	0.9670	0.9832	7	10	1.88
LSTM	FastText	Op	0.9921	0.9236	0.9593	7	10	1.50
LSTM	1-hot	Op	0.9998	0.9885	0.9942	7	10	1.42

For both the PARSEC-500 and PARSEC-5000 datasets, 1-hot encoding showed the best performance and indicated that, despite its simplicity, it is highly effective for our task. FastText appears to be the least effective among the tested representations, yielding the lowest F1 scores for both models. This might suggest that FastText’s sub-word features and simpler contextual understanding do not capture enough discriminating information for our specific dataset and task.

5.7.2. The Effect of Data Features

This section examines the top models’ performance on different feature sets, analyzed on the PARSEC-500 and PARSEC-5000 datasets (full results for additional features are available in Appendix A in Appendix A.7). Feature sets are described in Section 3.5. The results are presented in Table 8. We observed that, surprisingly, the best scores of all four models were achieved when the smaller feature set was selected (OpsRes for PARSEC-500 and Ops for PARSEC-5000). Moreover, adding process-duration features and process details reduced the sensitivity and F1 score drastically, implying that these features interfere with the abilities of neural models to detect ransomware. One possible reason is that these features introduce noise and may be correlated with existing features, leading to redundancy and diluting the impact of significant features. Additionally, increasing data

dimensionality makes learning more difficult for models if the new features do not carry substantial information relevant to the task.

Table 8. Scores of the top models on the PARSEC-500 and PARSEC-5000 datasets with different data features (the best scores are marked in gray).

PARSEC-500 dataset							
model	text repr	FLIST	sensitivity	specificity	F1	W	ep
CNN	1-hot	OpsRes	0.9978	0.9827	0.9903	7	10
CNN	1-hot	OpsRes	0.9978	0.9827	0.9903	7	10
CNN	1-hot	OpsResDur	0.6100	0.5818	0.6015	7	10
CNN	1-hot	OpsResDurDet	0.1127	0.9859	0.2000	7	10
PARSEC-5000 dataset							
model	text repr	FLIST	sensitivity	specificity	F1	W	ep
LSTM	1-hot	Ops	0.9998	0.9885	0.9942	7	10
LSTM	1-hot	OpsRes	1	0.9751	0.9877	7	10
LSTM	1-hot	OpsResDur	0.4085	0.8332	0.5186	7	10
LSTM	1-hot	OpsResDurDet	0.1127	0.9122	0.1877	7	10

5.7.3. The Effect of the API Call-Window Size

This section examines top models' performance on different API call-sequence sizes, analyzed on the PARSEC-500 dataset (full results are available in Appendix A in Appendices A.5 and A.6). We examined how the scores were affected by selecting window sizes of $W = 1, 3, 5, 7$. The results are presented in Table 9. Because the top models for both datasets have $W = 7$, we were interested in seeing how sharp the drop was in the F1 scores. We can see that the scores decreased steadily when the window size fell from 7 to 5 and 3, but the biggest decrease happened when the window size was set to 1. This is a clear indication of the need for neural models to have information on more than one consecutive system call for every process.

Table 9. Scores of the top models on the PARSEC-500 and PARSEC-5000 datasets with different window sizes (the best scores are marked in gray).

PARSEC-500 dataset								
model	text repr	FLIST	sensitivity	specificity	F1	W	ep	test time(s)
CNN	1-hot	OpsRes	0.9885	0.7017	0.8645	1	10	0.57
CNN	1-hot	OpsRes	0.9963	0.9101	0.9551	3	10	0.34
CNN	1-hot	OpsRes	0.9962	0.9431	0.9704	5	10	0.20
CNN	1-hot	OpsRes	0.9978	0.9827	0.9903	7	10	0.20
PARSEC-5000 dataset								
model	text repr	FLIST	sensitivity	specificity	F1	W	ep	test time(s)
LSTM	1-hot	Ops	0.9957	0.6741	0.8578	1	10	6.29
LSTM	1-hot	Ops	0.9971	0.8944	0.9484	3	10	2.55
LSTM	1-hot	Ops	0.9997	0.9628	0.9816	5	10	1.74
LSTM	1-hot	Ops	0.9998	0.9885	0.9942	7	10	1.42

5.7.4. Increasing the Number of Training Epochs

This section examines the top models' performance with different numbers of training epochs, analyzed on the PARSEC-500 and PARSEC-5000 datasets. We examined how the scores were affected by selecting the number of epochs to be $ep = 10, 20, 30$. Table 10 shows the results of this evaluation, including test and train times. We can see that the best performance was achieved for $ep = 10$ and that there was no need to increase the number of training epochs beyond that. This decision decreased the training time significantly, especially for the PARSEC-500 dataset. We also observed that the test times were not affected by increasing the number of training epochs.

Table 10. Scores of the top models on the PARSEC-500 and PARSEC-5000 datasets with a different number of training epochs (the best scores are marked in gray).

PARSEC-500 dataset									
model	text repr	FLIST	sens-ty	spec-ty	F1	W	ep	train time(s)	test time (s)
CNN	1-hot	OpsRes	0.9978	0.9827	0.9903	7	10	19.43	0.20
CNN	1-hot	OpsRes	0.9978	0.9751	0.9866	7	20	37.58	0.27
CNN	1-hot	OpsRes	0.9957	0.9805	0.9882	7	30	58.99	0.33
PARSEC-5000 dataset									
model	text repr	FLIST	sens-ty	spec-ty	F1	W	ep	train time(s)	test time (s)
LSTM	1-hot	Ops	0.9998	0.9885	0.9942	7	10	258.94	1.42
LSTM	1-hot	Ops	0.9991	0.9802	0.9898	7	20	507.38	1.30
LSTM	1-hot	Ops	1.0000	0.9837	0.9919	7	30	767.46	1.35

5.7.5. Different Train–Test Splits

Here, we present the results of our test for our top models' robustness by applying them to different train–test splits (described in Section 4.2) to the PARSEC-500 and PARSEC-5000 datasets. In these splits, different benign and ransomware processes were assigned to the train and test sets. Table 11 shows how selecting different processes during the train–test split affects the ratio of API call features unique to benign or ransomware processes. These calls help the models identify processes without the need for deep analysis. Additionally, we found the feature SetRenameInformationFile to be a unique ransomware feature that was recorded 1111 times exclusively in ransomware activities. This feature was not present in any of the benign processes.

Table 11. Benign processes' selection.

Split #	Benign Processes	Unique Features
-	all	57
1	CompatTelRunner.exe, smss.exe, wmpnetwk.exe, curl.exe, wsqm-cons.exe, powershell.exe, lame.exe, DllHost.exe, GoogleCrashHandler64.exe, Idle, taskhost.exe, libreofficeCalcTest.exe, soffice.bin	44
2	Idle, sppsvc.exe, VBoxTray.exe, csrss.exe, wmiprvse.exe, steam.exe, schtasks.exe, taskeng.exe, GoogleCrashHandler.exe, EXCEL.EXE, cmd.exe, curl.exe, helper.exe	35
3	sdclt.exe, lame.exe, SearchFilterHost.exe, ffmpeg.exe, Explorer.EXE, wmpnetwk.exe, PT-CPUtest64.exe, EXCEL.EXE, winlogon.exe, conhost.exe, compattelrunner.exe, Browsing.exe, lsm.exe, GoogleCrashHandler64.exe, DllHost.exe, AUDIODG.EXE, wmiprvse.exe, WordProcessing.exe, cmd.exe, sc.exe, csrss.exe, lame.exe, NativeApp.exe, DeviceDisplayObjectProvider.exe, spoolsv.exe, WMIADAP.EXE	37
4	Explorer.EXE, DiagTrackRunner.exe, taskhost.exe, wmiprvse.exe, sppsvc.exe, System, cmd.exe, NativeApp.exe, GoogleUpdate.exe, svchost.exe, schtasks.exe, soffice.bin, PT-BulletPhysics64.exe	34
5		44

Table 12 contains the results of the top models on the PARSEC-500 and PARSEC-5000 datasets with different data splits. We observed that sensitivity scores remained high or identical for both datasets and for different splits, but there was variability in the F1 scores on the PARSEC-500 dataset. The high or identical sensitivity scores across different splits suggest that the models were consistently good at identifying positive cases in both datasets, which indicates the models' robustness. The variability in F1 scores on the PARSEC-500 dataset implies that the choice of processes for the training set can significantly affect the models' performance in terms of precision and recall balance. However, the reduced

variability in F1 scores on the larger PARSEC-5000 dataset indicates that a larger dataset provides more stable and reliable performance, reducing the impact of specific training-set selections. We conclude that longer API call sequences in the PARSEC-5000 dataset led to successful ransomware detection, regardless of the training-set processes. This observation implies that more comprehensive data (longer sequences) enhance the models' robustness and reliability. For the smaller PARSEC-500 dataset, the selection of processes for the training set had a more pronounced effect on the models' performance. This suggests that, with limited data, the specific characteristics of the training set play a crucial role in determining the models' effectiveness. It highlights the importance of careful training set selection in low-data scenarios.

Table 12. Scores of the top models on the PARSEC-500 and PARSEC-5000 datasets with different train–test splits (the best scores are marked in gray).

PARSEC-5000 dataset									
model	text repr	FLIST	sensitivity	specificity	F1	test time (s)	W	ep	split
CNN	1-hot	OpsRes	0.9978	0.9827	0.9903	0.20	7	10	1
CNN	1-hot	OpsRes	0.9989	0.9881	0.9935	0.27	7	10	2
CNN	1-hot	OpsRes	0.9957	0.9675	0.9818	0.15	7	10	3
CNN	1-hot	OpsRes	0.9957	0.9827	0.9892	0.15	7	10	4
CNN	1-hot	OpsRes	0.9967	0.9621	0.9798	0.14	7	10	5
PARSEC-5000 dataset									
model	text repr	FLIST	sensitivity	specificity	F1	test time (s)	W	ep	split
LSTM	1-hot	Ops	0.9998	0.9885	0.9942	1.42	7	10	1
LSTM	1-hot	Ops	0.9998	0.9897	0.9947	1.38	7	10	2
LSTM	1-hot	Ops	0.9998	0.9887	0.9943	1.28	7	10	3
LSTM	1-hot	Ops	0.9998	0.9867	0.9933	1.29	7	10	4
LSTM	1-hot	Ops	0.9998	0.9884	0.9941	1.32	7	10	5

5.7.6. The Effect of Unbalanced Data

Table 13 illustrates the behavior of a binary classification model when evaluated on test sets with varying class ratios despite being trained on a balanced dataset. Interestingly, the F1 score rose with the percentage of class 1 (the RW class) in the test data. When class 1 was underrepresented, for example, at 1% of the test set, the F1 score was lower; nevertheless, as the distribution became more balanced, at 40% of the RW class, the F1 score increased dramatically. The model reliably identified positive and negative instances with high accuracy, maintaining exceptional sensitivity and specificity across all configurations despite these differences in F1 score. The models' fundamental capability to identify both classes was demonstrated by their consistency in sensitivity and specificity. This indicates that the core ability of our models to detect both classes remained strong, even as the class distribution in the test set shifted.

Table 13. Scores of the top models on the PARSEC-500 and PARSEC-5000 datasets with different test-set benign–ransomware ratios (the best scores are marked in gray).

PARSEC-500 dataset						
model	FLIST	benign–RW ratio	acc	sensitivity	specificity	F1
CNN 1-hot W=7 eps=10	OpsRes	99/1	0.9826	1.0000	0.9825	0.5294
CNN 1-hot W=7 eps=11	OpsRes	95/5	0.9826	1.0000	0.9817	0.8519
CNN 1-hot W=7 eps=12	OpsRes	90/10	0.9837	1.0000	0.9819	0.9246
CNN 1-hot W=7 eps=13	OpsRes	80/20	0.9870	1.0000	0.9837	0.9684
CNN 1-hot W=7 eps=14	OpsRes	70/30	0.9870	0.9964	0.9830	0.9786
CNN 1-hot W=7 eps=15	OpsRes	60/40	0.9913	1.0000	0.9855	0.9893

Table 13. Cont.

model	PARSEC-500 dataset					
	FLIST	benign–RW ratio	acc	sensitivity	specificity	F1
LSTM 1-hot W=7 eps=10	Ops	99/1	0.9872	1.0000	0.9870	0.6073
LSTM 1-hot W=7 eps=11	Ops	95/5	0.9875	1.0000	0.9868	0.8889
LSTM 1-hot W=7 eps=12	Ops	90/10	0.9880	0.9989	0.9868	0.9435
LSTM 1-hot W=7 eps=13	Ops	80/20	0.9898	1.0000	0.9872	0.9750
LSTM 1-hot W=7 eps=14	Ops	70/30	0.9909	0.9993	0.9874	0.9851
LSTM 1-hot W=7 eps=15	Ops	60/40	0.9927	1.0000	0.9878	0.9909

6. Conclusions

In this paper, we have explored the efficacy of deep learning techniques in the early detection of ransomware through the analysis of API call sequences. We designed and created a comprehensive dataset of initial API call sequences of popular benign processes and verified ransomware processes. We also performed a comprehensive analysis of different baseline and neural-network models applied to the task of ransomware detection on this dataset.

Our investigation has provided substantial evidence that neural network models, especially CNN and LSTM, can be effectively applied to differentiate between benign and malicious system behaviors. We demonstrated that these models outperform traditional ML classifiers (baselines) and a competing method of [29], providing a positive answer to RQ1. Our findings indicate that the inclusion of the result feature for each API call significantly improved the models' performance, providing a positive answer to RQ2. We also found that 1-hot encoding of text features yielded the best results, answering RQ3. We, moreover, learned that increasing the number W of consecutive API calls used in the analysis improved the classification accuracy and F1-measure and that setting $W = 7$ was sufficient to achieve state-of-the-art results.

Across various configurations, the combination of operation and result features yielded the best results. Additionally, our analysis showed that a window size of 7 provided optimal performance, and 1-hot encoding (OH) generally outperformed other encoding methods in terms of accuracy, answering RQ4. Finally, we learned that the test times of neural models are suitable for online ransomware detection, which resolves RQ5.

We hope the PARSEC dataset will become a valuable resource for the cybersecurity community and encourage further research in the area of ransomware detection. Our findings contribute to the development of more robust and efficient ransomware detection systems, advancing the field of cybersecurity.

7. Limitations and Future Research Directions

The findings of this paper open several directions for future research, namely (1) the expansion of the dataset to capture a broader spectrum of real user activities and (2) the exploration of real-time detection systems integrated into network infrastructures. The PARSEC dataset, while robust, primarily includes API call sequences from simulated benign and ransomware processes. There is a compelling need to develop a dataset that will include activities from diverse computing environments such as office tasks, multimedia processing, software development, and gaming. Current ransomware detection models largely operate by analyzing static datasets. However, integrating these models into live network systems could facilitate the detection of ransomware as it attempts to execute. This approach would enable a more dynamic and proactive response to ransomware threats.

The limitations of our approach are the challenges associated with using API call features and neural models for ransomware detection. Collecting and labeling a comprehensive dataset of API call sequences from benign and ransomware processes is complex, time-consuming, and resource-intensive. Maintaining dataset quality and relevance as ransomware evolves requires substantial effort and depends on the chosen processes. Neural models, particularly deep learning ones, risk overfitting specific patterns in the training data. This can result in recognizing only known ransomware sequences, rather than general malicious behavior, necessitating extensive and resource-heavy testing to ensure good

generalization. We also observed that the selection of processes for the training set had an effect on the performance of the model when shorter API call sequences were used as training data. This means that future applications should be mindful of this phenomenon.

Author Contributions: Conceptualization, M.K., M.D. and N.V.; methodology, M.K., M.D. and N.V.; software, M.D.; validation, M.D.; formal analysis, M.K., M.D. and N.V.; resources, M.K. and M.D.; data curation, M.D.; writing—original draft preparation, M.K. and N.V.; writing—review and editing, M.K., M.D. and N.V.; supervision, M.K. and N.V. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Data Availability Statement: The PARSEC dataset and the code reside in a public repository on GitHub. It is freely available to the community at <https://github.com/MatanDavidian/MSc-Ransomware-Detection-Using-Deep-Learning-Models>.

Conflicts of Interest: The authors declare no conflicts of interest.

Abbreviations

The following abbreviations are used in this manuscript:

API	Application programming interface
BERT	Bidirectional encoder representations from transformers
CNN	Convolutional neural network
CPU	Central processing unit
DGCNN	Deep graph convolutional neural network
DGNN	Deep graph neural network
DL	Deep learning
DNN	Deep neural network
F1	F1 measure
FPs	False positives
FNs	False negatives
GDDR5	Graphics double data rate type five
GPU	Graphics processing unit
IRP	I/O request packet
kNN	k-nearest neighbors
LSTM	Long short-term memory
LR	Logistic regression
ML	Machine learning
MLP	Multi-layer perceptron
NLP	Natural language processing
Ops	Operations
OpsRes	Operations with results
OpsResDur	Operations with results and duration
OpsResDurDet	Operations with results, duration, and details
OS	Operating system
P	Precision
PM	Process monitor
R	Recall
RaaS	Ransomware-as-a-service
RAM	Random access memory
RF	Random forest
RNN	Recurrent neural network
RQ	Research question
RW	Ransomware
SSD	Solid-state drive
SVM	Support vector machine
SE	Sentence embeddings
SP1	Service Pack 1

TPs	True positives
TNs	True negatives
VM	Virtual machine

Appendix A

Appendix A.1. Full List of Benign Processes

Table A1 lists the benign processes we used in the PARSEC dataset's construction.

Table A1. General benign processes.

Process Name	Category
General Processes	
AxCrypt.exe	encryption
ffmpeg.exe	multimedia
EXCEL.EXE	office
WinRAR.exe	compression
WINWORD.EXE	office
7zG.exe	compression
curl.exe	downloading
lame.exe	multimedia
benchmarking processes	
PerformanceTest64.exe	Pass Mark
PT-CPUtest64.exe	Pass Mark
PT-BulletPhysics64.exe	Pass Mark
soffice.bin	PC Mark
libreofficeCalcTest.exe	PC Mark
Browsing.exe	PC Mark
MFPlayback.exe	PC Mark
MFVideoChat2.exe	PC Mark
WordProcessing.exe	PC Mark
NativeApp.exe	PC Mark

Table A2 contains the full list of idle-state processes.

Table A2. Idle-state benign processes.

Process Names		
System	SearchIndexer.exe	Cobian.Reflector.UserInterface.exe
Idle	smss.exe	csrss.exe
wininit.exe	winlogon.exe	services.exe
lsass.exe	lsmd.exe	svchost.exe
VBoxService.exe	AUDIODG.EXE	spoolsv.exe
taskhost.exe	Cobian.Reflector.VSCRequester.exe	taskeng.exe
sppsvc.exe	Dwm.exe	Explorer.EXE
VBoxTray.exe	Cobian.Reflector.Application.exe	btweb.exe
steam.exe	helper.exe	wmpnetwk.exe
wmiprvse.exe	SearchProtocolHost.exe	SearchFilterHost.exe
cmd.exe	conhost.exe	powershell.exe
DllHost.exe	WMIADAP.EXE	IntelSoftwareAssetManagerService.exe
sc.exe	sdclt.exe	DiagTrackRunner.exe
wsqmcons.exe	schtasks.exe	CompatTelRunner.exe
GoogleUpdate.exe	GoogleCrashHandler.exe	GoogleCrashHandler64.exe
DeviceDisplayObjectProvider.exe	sdclt.exe	steam.exe
DeviceDisplayObjectProvider.exe	DiagTrackRunner.exe	helper.exe
GoogleCrashHandler.exe	compattelrunner.exe	GoogleUpdate.exe

Appendix A.2. Full Information on Process Parameters

Table A3 contains the full list of process details parameters used in the PARSEC datasets.

Table A3. List of process-detail parameters.

Parameters			
FileAttributes	DeletePending	Disposition	Options
Attributes	ShareMode	Access	Exclusive
FailImmediately	OpenResult	PageProtection	Control
ExitStatus	PrivateBytes	PeakPrivateBytes	WorkingSet
PeakWorkingSet	Commandline	Priority	GrantedAccess
Name	Type	Data	Query
HandleTags	I/OFlags	FileSystemAttributes	DesiredAccess

Table A4 contains the full list of process-detail parameters filtered out from the PAR-SEC dataset.

Table A4. List of unused process parameters.

Parameters			
PID	ID	connid	ChangeTime
CreationTime	LastAccessTime	LastWriteTime	Startime
Endtime	Time	VolumeCreationTime	Directory
FileName	Size	AllocationSize	EaSize
Environment	FileInformationClass	FileSystemName	Length
	MaximumComponentNameLength		

Table A5 contains the full list of process operation names used in our dataset.

Table A5. List of process operation names.

Parameters		
CloseFile	CreateFile	CreateFileMapping
DeviceIoControl	FASTIO_ACQUIRE_FOR_CC_FLUSH	
FASTIO_ACQUIRE_FOR_MOD_WRITE		
FASTIO_MDL_READ_COMPLETE		
FASTIO_MDL_WRITE_COMPLETE		
FASTIO_RELEASE_FOR_CC_FLUSH		
FASTIO_RELEASE_FOR_MOD_WRITE		
FASTIO_RELEASE_FOR_SECTION_SYNCHRONIZATION		
FileSystemControl	FlushBuffersFile	IRP_MJ_CLOSE
Load Image	LockFile	NotifyChangeDirectory
Process Create	Process Exit	Process Profiling
Process Start	QueryAllInformationFile	QueryAttributeInformationVolume
QueryAttributeTagFile	QueryBasicInformationFile	QueryDirectory
QueryEaInformationFile	QueryFileInternalInformationFile	QueryInformationVolume
QueryNameInformationFile	QueryNetworkOpenInformationFile	QueryNormalizedNameInformationFile
QueryOpen	QuerySecurityFile	QuerySizeInformationVolume
QueryStandardInformationFile	QueryStreamInformationFile	ReadFile
RegCloseKey	RegCreateKey	RegDeleteKey
RegDeleteValue	RegEnumKey	RegEnumValue
RegLoadKey	RegOpenKey	RegQueryKey
RegQueryKeySecurity	RegQueryMultipleValueKey	RegQueryValue
RegSetInfoKey	RegSetValue	SetAllocationInformationFile
SetBasicInformationFile	SetDispositionInformationFile	SetEndOfFileInformationFile
SetRenameInformationFile	SetSecurityFile	TCP Accept
TCP Connect	TCP Disconnect	TCP Receive
TCP Send	TCP TCPCopy	Thread Create
Thread Exit	UDP Receive	UDP Send
UnlockFileSingle	WriteFile	

Table A6 contains the full list of API call-result parameters used in our dataset.

Table A6. List of API call-result parameters.

Parameters	
SUCCESS	FILE LOCKED WITH ONLY READERS
FILE LOCKED WITH WRITERS	ACCESS DENIED
IS DIRECTORY	NAME COLLISION
NAME INVALID	NAME NOT FOUND
PATH NOT FOUND	REPARSE
SHARING VIOLATION	FAST IO DISALLOWED
INVALID PARAMETER	CANT WAIT
END OF FILE	INVALID DEVICE REQUEST
NOT REPARSE POINT	NOTIFY CLEANUP
BUFFER OVERFLOW	NO MORE FILES
NO SUCH FILE	NO MORE ENTRIES
BUFFER TOO SMALL	FILE LOCK CONFLICT

Appendix A.3. The Number of System Calls for Benign and Ransomware Processes

Tables A7 and A8 show the total numbers for API system calls for benign and ransomware processes for the PARSEC-500 and PARSEC-5000 datasets.

Table A7. Comparison of system call amounts for the PARSEC-500 dataset.

Operation	Benign	Ransomware
CloseFile	1168	5052
CreateFile	1921	5061
CreateFileMapping	1774	0
FASTIO_ACQUIRE_FOR_CC_FLUSH	177	0
FASTIO_ACQUIRE_FOR_MOD_WRITE	58	0
FASTIO_RELEASE_FOR_CC_FLUSH	176	0
FASTIO_RELEASE_FOR_MOD_WRITE	52	0
FASTIO_RELEASE_FOR_SECTION_SYNCHRONIZATION	1540	0
FileSystemControl	298	0
IRP_MJ_CLOSE	915	3717
Load Image	543	0
Process Create	5	0
Process Exit	4	0
Process Profiling	6402	228
Process Start	34	0
QueryAllInformationFile	6	0
QueryAttributeInformationVolume	15	0
QueryAttributeTagFile	514	1269
QueryBasicInformationFile	327	3306
QueryDirectory	153	601
QueryFileInternalInformationFile	777	0
QueryInformationVolume	36	0
QueryNameInformationFile	131	0
QueryNetworkOpenInformationFile	96	0
QueryNormalizedNameInformationFile	2	0
QueryOpen	388	1075
QuerySecurityFile	62	0
QueryStandardInformationFile	678	1230
ReadFile	1618	2888
RegCloseKey	1278	0
RegCreateKey	32	0
RegDeleteKey	2	0
RegDeleteValue	18	0
RegEnumKey	253	0
RegEnumValue	77	0
RegOpenKey	2617	0
RegQueryKey	1546	0
RegQueryKeySecurity	56	0
RegQueryMultipleValueKey	8	0
RegQueryValue	1872	0
RegSetInfoKey	173	0
RegSetValue	10	0
SetBasicInformationFile	780	1123

Table A7. *Cont.*

Operation	Benign	Ransomware
SetEndOfFileInformationFile	4	0
SetRenameInformationFile	0	1259
TCP Connect	1	0
TCP Receive	8	0
TCP Send	3	0
Thread Create	168	0
Thread Exit	132	0
UDP Receive	5	0
WriteFile	216	2606

Table A8. Comparison of system-call amounts for the PARSEC-5000 dataset.

Operation	Benign	Ransomware
CloseFile	9151	22,323
CreateFile	11,044	23,374
CreateFileMapping	7831	0
DeviceIoControl	48	0
FASTIO_ACQUIRE_FOR_CC_FLUSH	1634	0
FASTIO_ACQUIRE_FOR_MOD_WRITE	735	0
FASTIO_MDL_READ_COMPLETE	5	0
FASTIO_MDL_WRITE_COMPLETE	5	0
FASTIO_RELEASE_FOR_CC_FLUSH	1634	0
FASTIO_RELEASE_FOR_MOD_WRITE	731	0
FASTIO_RELEASE_FOR_SECTION_SYNCHRONIZATION	6560	0
FileSystemControl	466	0
FlushBuffersFile	2	0
IRP_MJ_CLOSE	6975	17,031
Load Image	2059	0
LockFile	2	0
NotifyChangeDirectory	1	0
Process Create	34	0
Process Exit	38	0
Process Profiling	8996	388
Process Start	56	0
QueryAllInformationFile	477	0
QueryAttributeInformationVolume	213	0
QueryAttributeTagFile	1189	4645
QueryBasicInformationFile	2210	15,182
QueryDirectory	1916	4546
QueryEaInformationFile	10	0
QueryFileInternalInformationFile	1469	0
QueryInformationVolume	619	0
QueryNameInformationFile	1289	0
QueryNetworkOpenInformationFile	1385	0
QueryNormalizedNameInformationFile	2	0
QueryOpen	3636	6987
QuerySecurityFile	559	0
QuerySizeInformationVolume	1	0
QueryStandardInformationFile	2443	4635
QueryStreamInformationFile	10	0
ReadFile	19,005	10,593
RegCloseKey	13,207	0
RegCreateKey	351	0
RegDeleteKey	9	0
RegDeleteValue	34	0
RegEnumKey	1806	0
RegEnumValue	836	0
RegLoadKey	7	0
RegOpenKey	24,754	0
RegQueryKey	14,763	0
RegQueryKeySecurity	452	0
RegQueryMultipleValueKey	34	0

Table A8. Cont.

Operation	Benign	Ransomware
RegQueryValue	24,298	0
RegSetInfoKey	1079	0
RegSetValue	70	0
SetAllocationInformationFile	2	0
SetBasicInformationFile	1496	4992
SetDispositionInformationFile	4	0
SetEndOfFileInformationFile	75	0
SetRenameInformationFile	3	4643
SetSecurityFile	8	0
TCP Accept	2	0
TCP Connect	3	0
TCP Disconnect	4	0
TCP Receive	2022	0
TCP Send	7	0
TCP TCPCopy	5	0
Thread Create	473	0
Thread Exit	343	0
UDP Receive	41	0
UDP Send	19	0
UnlockFileSingle	2	0
WriteFile	5251	9986

Appendix A.4. Operations' Visualization for Benign and Ransomware Processes

Figures A1–A5 show the distribution of operational API calls for various benign processes.

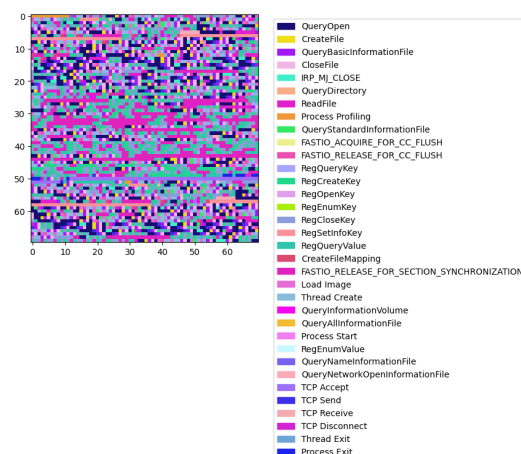


Figure A1. AxCrypt.exe (encryption tool).

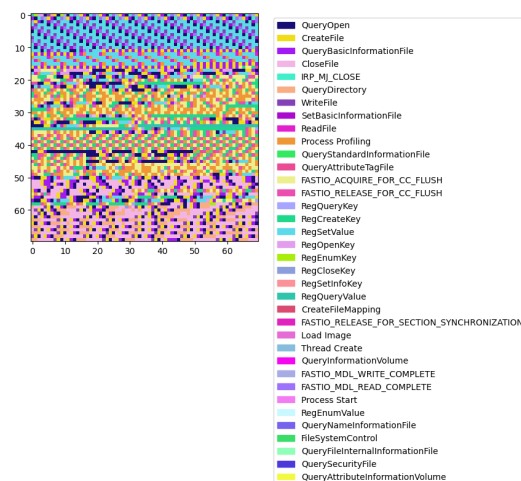


Figure A2. WinRAR.exe (file compression and archiving).

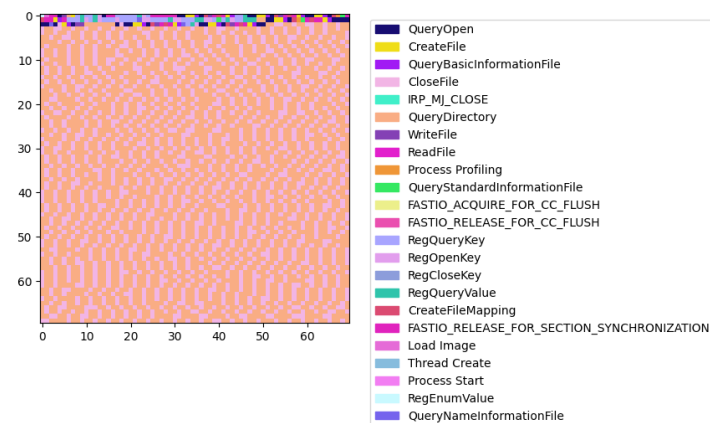


Figure A3. *lame.exe* (audio encoder).

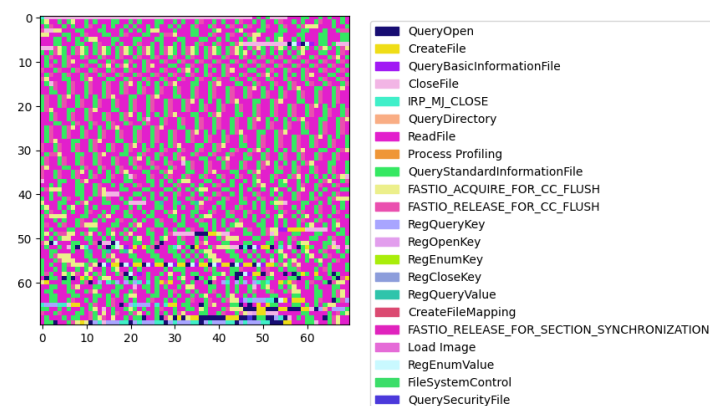


Figure A4. *Explorer.EXE* (file management and navigation).

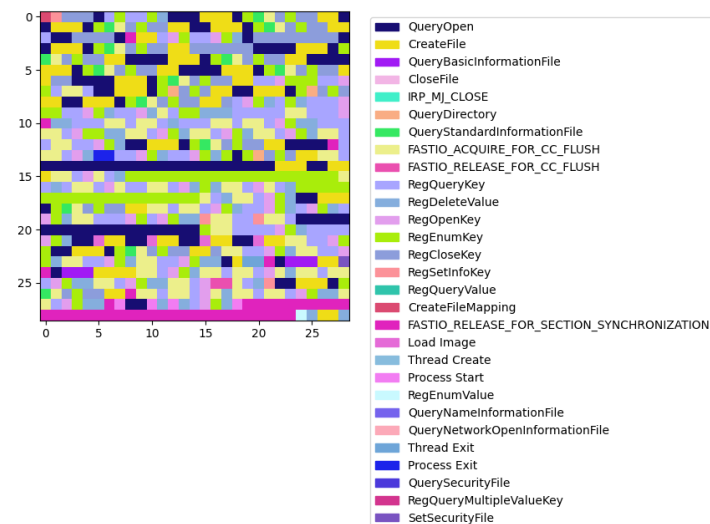


Figure A5. *GoogleCrashHandler.exe* (crash reporting service).

Figures A6–A10 show the distribution of operational API calls for various ransomware process types.

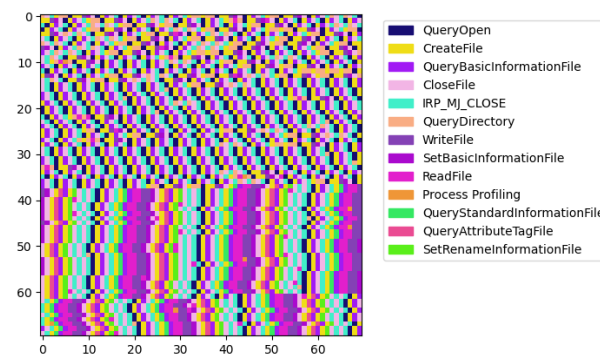


Figure A6. 3359dff8c8b3855e8cf980539e7fb300.exe (ransomware sample).

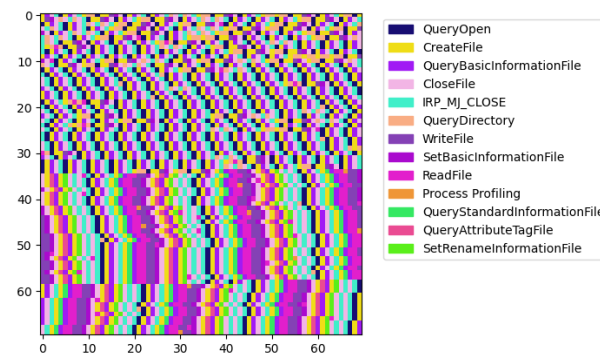


Figure A7. 0b7fa305b57066885d7d70c96d51aae0.exe (ransomware sample).

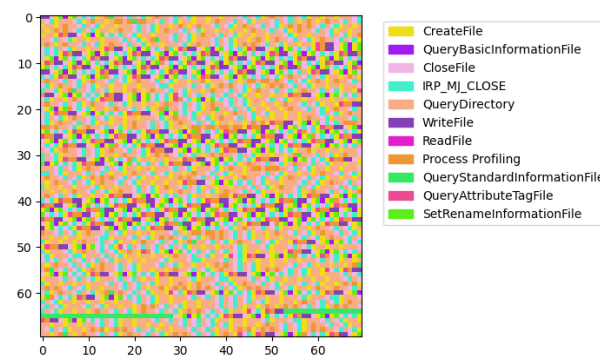


Figure A8. 1a4bf948ba5876657cde4ea846e13f74.exe (ransomware sample).

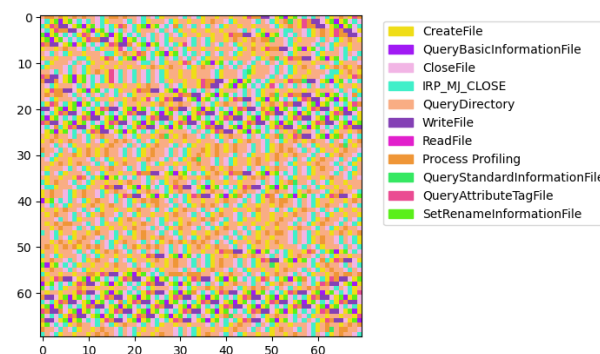


Figure A9. 6e20f33646814a547b1d6a9b55343e38.exe (ransomware sample).

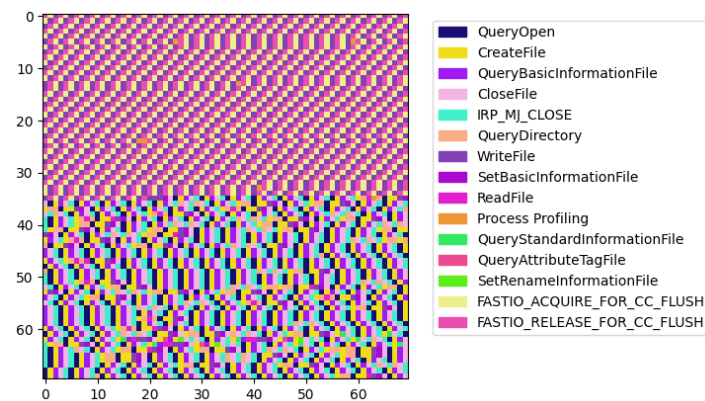


Figure A10. 66b7a800f6a7f327de0eed42407074ce.exe (ransomware sample).

Appendix A.5. Full Experimental Results for the MLP Model

Tables A9 and A10 contain full evaluation results for the MLP model on the PARSEC-500 and PARSEC-5000 datasets. This evaluation shows all feature combinations (Ops, OpsRes, OpsResDur, and OpsResDurDet), all text representations (1-hot encoding, FastText word vectors, and BERT sentence embeddings), and all window sizes (1, 3, 5, and 7). We report sensitivity, specificity, and F1 measures for every data setup. Note that, for the two large feature sets, 1-hot results are not reported because the vectors were too large for the model to be trained within a reasonable amount of time. The data reveal a pattern of increasing scores for all the datasets with the best results achieved for $W = 7$. There was a significant boost in performance when the sequence size was increased from 1 to 3, and further increases in sequence size yielded progressively smaller improvements. The test time remained small, and its variations suggest that, while larger window sizes typically increase the computational time, the effect is not uniformly significant across all API call amounts.

Table A9. Full experimental results for the MLP model on the PARSEC-500 dataset.

Model	Text repr	FLIST	Sensitivity	Specificity	F1	W	
MLP	1-hot	Op	0.9923	0.6682	0.8539	0.01	1
MLP	1-hot	Op	0.9954	0.8953	0.9479	0.00	3
MLP	1-hot	Op	0.9977	0.9169	0.9590	0.00	5
MLP	1-hot	Op	0.9967	0.9458	0.9720	0.00	7
MLP	BERT SE	Op	0.9929	0.6691	0.8546	0.04	1
MLP	BERT SE	Op	0.9958	0.8934	0.9473	0.03	3
MLP	BERT SE	Op	0.9969	0.8946	0.9484	0.03	5
MLP	BERT SE	Op	0.9935	0.9664	0.9802	0.03	7
MLP	FastText	Op	0.9086	0.5474	0.7696	0.02	1
MLP	FastText	Op	0.9634	0.6951	0.8494	0.02	3
MLP	FastText	Op	0.9223	0.8238	0.8790	0.02	5
MLP	FastText	Op	0.9480	0.7779	0.8737	0.01	7
MLP	1-hot	OpRes	0.9885	0.7017	0.8645	0.01	1
MLP	1-hot	OpRes	0.9958	0.8994	0.9500	0.01	3
MLP	1-hot	OpRes	0.9969	0.9346	0.9668	0.00	5
MLP	1-hot	OpRes	0.9946	0.9567	0.9761	0.00	7
MLP	BERT SE	OpRes	0.9892	0.6837	0.8581	0.08	1
MLP	BERT SE	OpRes	0.9949	0.8601	0.9321	0.07	3
MLP	BERT SE	OpRes	0.9977	0.8954	0.9491	0.07	5
MLP	BERT SE	OpRes	0.9967	0.9372	0.9679	0.13	7
MLP	FastText	OpRes	0.9455	0.5994	0.8060	0.10	1

Table A9. Cont.

Model	Text repr	FLIST	Sensitivity	Specificity	F1	W	
MLP	FastText	OpRes	0.9713	0.7502	0.8746	0.03	3
MLP	FastText	OpRes	0.9715	0.8292	0.9070	0.03	5
MLP	FastText	OpRes	0.9317	0.8646	0.9015	0.03	7
MLP	BERT SE	OpResDur	0.9892	0.6858	0.8589	0.09	1
MLP	BERT SE	OpResDur	0.9921	0.8703	0.9351	0.18	3
MLP	BERT SE	OpResDur	1.0000	0.8931	0.9493	0.19	5
MLP	BERT SE	OpResDur	0.9805	0.9372	0.9597	0.09	7
MLP	FastText	OpResDur	0.9472	0.5988	0.8067	0.05	1
MLP	FastText	OpResDur	0.9685	0.7790	0.8847	0.04	3
MLP	FastText	OpResDur	0.9792	0.7977	0.8977	0.03	5
MLP	FastText	OpResDur	0.9653	0.8234	0.9014	0.03	7
MLP	BERT SE	OpResDurDet	0.9851	0.7212	0.8703	0.47	1
MLP	BERT SE	OpResDurDet	0.9972	0.8800	0.9420	0.48	3
MLP	BERT SE	OpResDurDet	0.9823	0.9123	0.9491	0.48	5
MLP	BERT SE	OpResDurDet	0.9827	0.9317	0.9583	0.47	7
MLP	FastText	OpResDurDet	0.9409	0.6438	0.8192	0.20	1
MLP	FastText	OpResDurDet	0.9856	0.8499	0.9230	0.21	3
MLP	FastText	OpResDurDet	0.9892	0.8808	0.9383	0.18	5
MLP	FastText	OpResDurDet	0.9653	0.9296	0.9484	0.18	7

Table A10. Full experimental results for the MLP model on the PARSEC-5000 dataset.

Model	Text repr	FLIST	Sensitivity	Specificity	F1	W	
MLP	1-hot	Op	0.9957	0.6741	0.8578	0.18	1
MLP	1-hot	Op	0.9974	0.8947	0.9487	0.05	3
MLP	1-hot	Op	0.9999	0.9479	0.9746	0.04	5
MLP	1-hot	Op	0.9989	0.9801	0.9896	0.04	7
MLP	BERT SE	Op	0.9952	0.6769	0.8586	11.02	1
MLP	BERT SE	Op	0.9979	0.8843	0.9443	0.31	3
MLP	BERT SE	Op	0.9990	0.9530	0.9765	0.40	5
MLP	BERT SE	Op	0.9995	0.9668	0.9834	0.34	7
MLP	FastText	Op	0.8732	0.6354	0.7804	3.67	1
MLP	FastText	Op	0.9850	0.8179	0.9090	0.14	3
MLP	FastText	Op	0.9828	0.8889	0.9388	0.15	5
MLP	FastText	Op	0.9915	0.8991	0.9477	0.13	7
MLP	1-hot	OpRes	0.9882	0.7103	0.8676	0.19	1
MLP	1-hot	OpRes	0.9988	0.9205	0.9612	0.07	3
MLP	1-hot	OpRes	0.9997	0.9538	0.9773	0.06	5
MLP	1-hot	OpRes	0.9999	0.9849	0.9925	0.05	7
MLP	FastText	OpRes	0.9888	0.6386	0.8415	2.53	1
MLP	FastText	OpRes	0.9919	0.7921	0.9018	0.24	3
MLP	FastText	OpRes	0.9837	0.9118	0.9495	0.23	5
MLP	FastText	OpRes	0.9906	0.9446	0.9684	0.25	7
MLP	FastText	OpResDur	0.9864	0.6349	0.8390	2.61	1
MLP	FastText	OpResDur	0.9964	0.7795	0.8989	0.37	3
MLP	FastText	OpResDur	0.9793	0.9111	0.9470	0.35	5
MLP	FastText	OpResDur	0.9583	0.9490	0.9539	0.38	7

Appendix A.6. Full Experimental Results for Neural Models

Tables A11 and A12 contain full evaluation results for the neural models (DNN, CNN, and LSTM) on the PARSEC-500 and PARSEC-5000 datasets. This evaluation shows all feature combinations, all text representations, and all window sizes. We report sensitivity, specificity, and F1 measures for every data setup.

Table A11. Full experimental results of neural models on the PARSEC-500 dataset.

Model	Text repr	FLIST	Sensitivity	Specificity	F1	W
DNN	BERT SE	Ops	0.9929	0.6615	0.8518	1
DNN	BERT SE	Ops	0.9981	0.8860	0.9452	3
DNN	BERT SE	Ops	1.0000	0.8431	0.9272	5
DNN	BERT SE	Ops	0.9859	0.9567	0.9717	7
DNN	FastText	Ops	0.9928	0.4828	0.7911	1
DNN	FastText	Ops	0.9819	0.7340	0.8736	3
DNN	FastText	Ops	0.9638	0.8323	0.9044	5
DNN	FastText	Ops	0.9426	0.8949	0.9206	7
DNN	1-hot	Ops	0.9923	0.6682	0.8539	1
DNN	1-hot	Ops	0.9954	0.8726	0.9378	3
DNN	1-hot	Ops	0.9977	0.9331	0.9665	5
DNN	1-hot	Ops	0.9935	0.9437	0.9693	7
DNN	BERT SE	OpsRes	0.9892	0.6791	0.8564	1
DNN	BERT SE	OpsRes	0.9838	0.8582	0.9257	3
DNN	BERT SE	OpsRes	0.9969	0.8877	0.9453	5
DNN	BERT SE	OpsRes	0.9978	0.9220	0.9614	7
DNN	FastText	OpsRes	0.9062	0.6212	0.7932	1
DNN	FastText	OpsRes	0.8828	0.8605	0.8731	3
DNN	FastText	OpsRes	0.9731	0.8523	0.9177	5
DNN	FastText	OpsRes	0.9567	0.8917	0.9265	7
DNN	1-hot	OpsRes	0.9885	0.7017	0.8645	1
DNN	1-hot	OpsRes	0.9949	0.9022	0.9508	3
DNN	1-hot	OpsRes	1.0000	0.9315	0.9669	5
DNN	1-hot	OpsRes	0.9967	0.9599	0.9787	7
DNN	BERT SE	OpsResDur	0.9892	0.6852	0.8587	1
DNN	BERT SE	OpsResDur	0.9991	0.8462	0.9281	3
DNN	BERT SE	OpsResDur	0.9923	0.8969	0.9471	5
DNN	BERT SE	OpsResDur	0.9913	0.9285	0.9611	7
DNN	FastText	OpsResDur	0.9897	0.6445	0.8440	1
DNN	FastText	OpsResDur	0.9286	0.8137	0.8782	3
DNN	FastText	OpsResDur	0.9685	0.8692	0.9227	5
DNN	FastText	OpsResDur	0.9632	0.8938	0.9309	7
DNN	BERT SE	OpsResDurDet	0.9851	0.7212	0.8703	1
DNN	BERT SE	OpsResDurDet	0.9981	0.8786	0.9418	3
DNN	BERT SE	OpsResDurDet	0.9946	0.9115	0.9549	5
DNN	BERT SE	OpsResDurDet	0.9686	0.9534	0.9613	7
DNN	FastText	OpsResDurDet	0.9788	0.6851	0.8534	1
DNN	FastText	OpsResDurDet	0.9875	0.8633	0.9298	3
DNN	FastText	OpsResDurDet	0.9823	0.9038	0.9452	5
DNN	FastText	OpsResDurDet	0.9729	0.9339	0.9543	7
CNN	BERT SE	Ops	0.9929	0.6691	0.8546	1
CNN	BERT SE	Ops	0.9981	0.8661	0.9363	3
CNN	BERT SE	Ops	0.9954	0.9085	0.9539	5
CNN	BERT SE	Ops	0.9859	0.9653	0.9759	7
CNN	FastText	Ops	0.9928	0.5282	0.8056	1
CNN	FastText	Ops	0.9690	0.7335	0.8669	3
CNN	FastText	Ops	0.9600	0.8677	0.9176	5
CNN	FastText	Ops	0.9599	0.8657	0.9167	7
CNN	1-hot	Ops	0.9923	0.6682	0.8539	1
CNN	1-hot	Ops	0.9991	0.8689	0.9380	3
CNN	1-hot	Ops	0.9962	0.9177	0.9585	5
CNN	1-hot	Ops	0.9989	0.9437	0.9721	7
CNN	BERT SE	OpsRes	0.9892	0.6791	0.8564	1
CNN	BERT SE	OpsRes	0.9930	0.8638	0.9328	3
CNN	BERT SE	OpsRes	0.9985	0.8969	0.9502	5
CNN	BERT SE	OpsRes	0.9989	0.9296	0.9654	7
CNN	FastText	OpsRes	0.9891	0.5837	0.8224	1
CNN	FastText	OpsRes	0.9930	0.7859	0.8999	3
CNN	FastText	OpsRes	0.9808	0.8631	0.9263	5
CNN	FastText	OpsRes	0.9610	0.8787	0.9230	7
CNN	1-hot	OpsRes	0.9885	0.7017	0.8645	1
CNN	1-hot	OpsRes	0.9963	0.9101	0.9551	3
CNN	1-hot	OpsRes	0.9962	0.9431	0.9704	5
CNN	1-hot	OpsRes	0.9978	0.9827	0.9903	7
CNN	BERT SE	OpsResDur	0.9892	0.6858	0.8589	1
CNN	BERT SE	OpsResDur	0.9995	0.8165	0.9157	3
CNN	BERT SE	OpsResDur	0.9615	0.9108	0.9377	5

Table A11. Cont.

Model	Text repr	FLIST	Sensitivity	Specificity	F1	W
CNN	BERT SE	OpsResDur	0.9881	0.9382	0.9641	7
CNN	FastText	OpsResDur	0.9862	0.5845	0.8212	1
CNN	FastText	OpsResDur	0.9801	0.7878	0.8941	3
CNN	FastText	OpsResDur	0.9708	0.8762	0.9269	5
CNN	FastText	OpsResDur	0.9707	0.8743	0.9261	7
CNN	BERT SE	OpsResDurDet	0.9840	0.7214	0.8698	1
CNN	BERT SE	OpsResDurDet	0.9977	0.8837	0.9439	3
CNN	BERT SE	OpsResDurDet	0.9969	0.9108	0.9558	5
CNN	BERT SE	OpsResDurDet	0.9978	0.9307	0.9654	7
CNN	FastText	OpsResDurDet	0.9405	0.6423	0.8184	1
CNN	FastText	OpsResDurDet	0.9903	0.8415	0.9217	3
CNN	FastText	OpsResDurDet	0.9015	0.9231	0.9114	5
CNN	FastText	OpsResDurDet	0.9729	0.8917	0.9349	7
LSTM	BERT SE	Ops	0.9929	0.6691	0.8546	1
LSTM	BERT SE	Ops	0.9972	0.8587	0.9326	3
LSTM	BERT SE	Ops	0.9962	0.9154	0.9575	5
LSTM	BERT SE	Ops	0.9978	0.9447	0.9720	7
LSTM	FastText	Ops	0.9486	0.6042	0.8092	1
LSTM	FastText	Ops	0.9815	0.7590	0.8832	3
LSTM	FastText	Ops	0.9838	0.7915	0.8975	5
LSTM	FastText	Ops	0.9653	0.8440	0.9101	7
LSTM	1-hot	Ops	0.9923	0.6682	0.8539	1
LSTM	1-hot	Ops	0.9893	0.8624	0.9303	3
LSTM	1-hot	Ops	0.9977	0.9123	0.9568	5
LSTM	1-hot	Ops	0.9957	0.9426	0.9699	7
LSTM	BERT SE	OpsRes	0.9892	0.6837	0.8581	1
LSTM	BERT SE	OpsRes	0.9944	0.8605	0.9320	3
LSTM	BERT SE	OpsRes	0.9992	0.9046	0.9541	5
LSTM	BERT SE	OpsRes	0.9935	0.9339	0.9648	7
LSTM	FastText	OpsRes	0.9891	0.6468	0.8445	1
LSTM	FastText	OpsRes	0.9435	0.8540	0.9031	3
LSTM	FastText	OpsRes	0.9738	0.8792	0.9299	5
LSTM	FastText	OpsRes	0.9632	0.9047	0.9358	7
LSTM	1-hot	OpsRes	0.9885	0.7017	0.8645	1
LSTM	1-hot	OpsRes	0.9907	0.8675	0.9332	3
LSTM	1-hot	OpsRes	1.0000	0.9146	0.9591	5
LSTM	1-hot	OpsRes	1.0000	0.9751	0.9877	7
LSTM	BERT SE	OpsResDur	0.9892	0.6858	0.8589	1
LSTM	BERT SE	OpsResDur	0.9861	0.8536	0.9248	3
LSTM	BERT SE	OpsResDur	0.9992	0.9023	0.9530	5
LSTM	BERT SE	OpsResDur	0.9978	0.9350	0.9674	7
LSTM	FastText	OpsResDur	0.9894	0.6445	0.8439	1
LSTM	FastText	OpsResDur	0.9300	0.8415	0.8906	3
LSTM	FastText	OpsResDur	0.9854	0.8715	0.9323	5
LSTM	FastText	OpsResDur	0.9686	0.8927	0.9332	7
LSTM	BERT SE	OpsResDurDet	0.9851	0.7212	0.8703	1
LSTM	BERT SE	OpsResDurDet	0.9972	0.8948	0.9486	3
LSTM	BERT SE	OpsResDurDet	0.9954	0.9231	0.9607	5
LSTM	BERT SE	OpsResDurDet	0.9924	0.9404	0.9673	7
LSTM	FastText	OpsResDurDet	0.9017	0.7185	0.8260	1
LSTM	FastText	OpsResDurDet	0.9917	0.8749	0.9370	3
LSTM	FastText	OpsResDurDet	0.9769	0.9069	0.9439	5
LSTM	FastText	OpsResDurDet	0.9729	0.9231	0.9493	7

Table A12. Full experimental results of neural models on the PARSEC-5000 dataset.

Model	Text repr	FLIST	Sensitivity	Specificity	F1	W
DNN	1-hot	Op	0.9957	0.6746	0.858	1
DNN	1-hot	Op	0.9981	0.8948	0.9494	3
DNN	1-hot	Op	0.9993	0.9610	0.9808	5
DNN	1-hot	Op	0.9992	0.9779	0.9890	7
DNN	BERT SE	Op	0.9952	0.6773	0.8676	1
DNN	BERT SE	Op	0.9980	0.8973	0.9531	3
DNN	BERT SE	Op	0.9998	0.9529	0.9773	5
DNN	BERT SE	Op	0.9998	0.9509	0.9760	7

Table A12. Cont.

Model	Text repr	FLIST	Sensitivity	Specificity	F1	W
DNN	FastText	Op	0.9956	0.6334	0.8500	1
DNN	FastText	Op	0.9929	0.8100	0.9129	3
DNN	FastText	Op	0.9787	0.8878	0.9364	5
DNN	FastText	Op	0.9482	0.9304	0.9443	7
DNN	1-hot	OpRes	0.9882	0.7104	0.8798	1
DNN	1-hot	OpRes	0.9990	0.9088	0.9593	3
DNN	1-hot	OpRes	0.9999	0.9626	0.9823	5
DNN	1-hot	OpRes	0.9997	0.9870	0.9942	7
DNN	FastText	OpRes	0.9939	0.6048	0.8331	1
DNN	FastText	OpRes	0.9655	0.7880	0.8961	3
DNN	FastText	OpRes	0.9894	0.9172	0.9556	5
DNN	FastText	OpRes	0.9829	0.9337	0.9606	7
DNN	FastText	OpResDur	0.9906	0.6123	0.8390	1
DNN	FastText	OpResDur	0.9941	0.7910	0.9027	3
DNN	FastText	OpResDur	0.9952	0.8967	0.9487	5
DNN	FastText	OpResDur	0.9856	0.9210	0.9549	7
CNN	1-hot	Op	0.9957	0.6747	0.8586	1
CNN	1-hot	Op	0.9983	0.8938	0.9488	3
CNN	1-hot	Op	0.9998	0.9573	0.9805	5
CNN	1-hot	Op	0.9988	0.9729	0.9887	7
CNN	BERT SE	Op	0.9952	0.6773	0.8587	1
CNN	BERT SE	Op	0.9950	0.8977	0.9491	3
CNN	BERT SE	Op	0.9988	0.9508	0.9759	5
CNN	BERT SE	Op	0.9988	0.9790	0.9896	7
CNN	FastText	Op	0.9384	0.5938	0.8108	1
CNN	FastText	Op	0.9944	0.7750	0.8989	3
CNN	FastText	Op	0.9564	0.9138	0.9388	5
CNN	FastText	Op	0.9836	0.9025	0.9462	7
CNN	1-hot	OpRes	0.9882	0.7103	0.8677	1
CNN	1-hot	OpRes	0.9959	0.9223	0.9612	3
CNN	1-hot	OpRes	0.9995	0.9645	0.9832	5
CNN	1-hot	OpRes	0.9972	0.9890	0.9934	7
CNN	FastText	OpRes	0.9888	0.6402	0.8429	1
CNN	FastText	OpRes	0.9417	0.8033	0.8868	3
CNN	FastText	OpRes	0.9845	0.8718	0.9361	5
CNN	FastText	OpRes	0.9703	0.9630	0.9684	7
CNN	FastText	OpResDur	0.9948	0.6300	0.8415	1
CNN	FastText	OpResDur	0.9933	0.7926	0.9090	3
CNN	FastText	OpResDur	0.9957	0.8912	0.9470	5
CNN	FastText	OpResDur	0.9978	0.9409	0.9746	7
LSTM	1-hot	Op	0.9957	0.6741	0.8580	1
LSTM	1-hot	Op	0.9971	0.8944	0.9485	3
LSTM	1-hot	Op	0.9997	0.9628	0.9816	5
LSTM	1-hot	Op	0.9998	0.9885	0.9940	7
LSTM	BERT SE	Op	0.9952	0.6772	0.8587	1
LSTM	BERT SE	Op	0.9973	0.8963	0.9495	3
LSTM	BERT SE	Op	0.9997	0.9511	0.9765	5
LSTM	BERT SE	Op	0.9988	0.9670	0.9834	7
LSTM	FastText	Op	0.9956	0.5618	0.8320	1
LSTM	FastText	Op	0.9360	0.8082	0.8808	3
LSTM	FastText	Op	0.9944	0.9077	0.9539	5
LSTM	FastText	Op	0.9921	0.9236	0.9593	7
LSTM	1-hot	OpRes	0.9882	0.7103	0.8676	1
LSTM	1-hot	OpRes	0.9986	0.9225	0.9668	3
LSTM	1-hot	OpRes	0.9997	0.9611	0.9816	5
LSTM	1-hot	OpRes	1.0000	0.9857	0.9931	7
LSTM	FastText	OpRes	0.9888	0.6622	0.8578	1
LSTM	FastText	OpRes	0.9944	0.8159	0.9215	3
LSTM	FastText	OpRes	0.9965	0.9188	0.9593	5
LSTM	FastText	OpRes	0.9907	0.9800	0.9860	7
LSTM	FastText	OpResDur	0.9895	0.6332	0.8413	1
LSTM	FastText	OpResDur	0.9962	0.8341	0.9319	3
LSTM	FastText	OpResDur	0.9980	0.9092	0.9559	5
LSTM	FastText	OpResDur	0.9936	0.9419	0.9702	7

Appendix A.7. Full Experimental Results for Neural Models—Additional Features

Table A13 shows the results of the neural-model evaluation for the feature sets OpResDur and OpResDurDet on the PARSEC-5000 and PARSEC-5000 datasets.

Table A13. Full experimental results of neural models on the PARSEC-5000 and PARSEC-5000 datasets with additional process features.

PARSEC-500 dataset							
model	text repr	FLIST	sensitivity	specificity	F1	W	ep
CNN	1-hot	OpResDur	0.6100	0.5818	0.6015	7	10
DNN	1-hot	OpResDur	0.2080	0.9458	0.3296	7	10
LSTM	1-hot	OpResDur	0.4085	0.8332	0.5186	7	10
MLP	1-hot	OpResDur	0.0195	0.9989	0.0382	7	10
PARSEC-5000 dataset							
model	text repr	FLIST	sensitivity	specificity	F1	W	ep
CNN	1-hot	OpResDurDet	0.1127	0.9859	0.2000	7	10
DNN	1-hot	OpResDurDet	0.0574	0.9957	0.1082	7	10
LSTM	1-hot	OpResDurDet	0.1127	0.9122	0.1877	7	10
MLP	1-hot	OpResDurDet	0.0184	0.9989	0.0361	7	10

References

- Cloudflare Inc. (n.d.) Cloudflare. What Is Ransomware? 2024. Available online: <https://www.cloudflare.com> (accessed on 1 August 2024).
- CrowdStrike. 2024 Global Threat Report. 2024. Available online: <https://www.crowdstrike.com> (accessed on 1 August 2024).
- Urooj, U.; Al-rimy, B.A.S.; Zainal, A.; Ghaleb, F.A.; Rassam, M.A. Ransomware detection using the dynamic analysis and machine learning: A survey and research directions. *Appl. Sci.* **2021**, *12*, 172. [\[CrossRef\]](#)
- Morgan, S. Ransomware deployment methods and analysis: Views from a predictive model and human responses. *Crime Sci. J.* **2021**, *10*, 2.
- Herrera Silva, J.A.; Barona López, L.I.; Valdivieso Caraguay, Á.L.; Hernández-Álvarez, M. A survey on situational awareness of ransomware attacks—Detection and prevention parameters. *Remote Sens.* **2019**, *11*, 1168. [\[CrossRef\]](#)
- McDonald, G.; Papadopoulos, P.; Pitropakis, N.; Ahmad, J.; Buchanan, W.J. Ransomware: Analysing the impact on Windows active directory domain services. *Sensors* **2022**, *22*, 953. [\[CrossRef\]](#)
- Zimba, A.; Chishimba, M. Analyzing the Impact of Ransomware Attacks Globally. *J. Cybersecur. Digit. Forensics* **2019**, *11*, 26.
- Zimba, A.; Chishimba, M. On the economic impact of crypto-ransomware attacks: The state of the art on enterprise systems. *Eur. J. Secur. Res.* **2019**, *4*, 3–31. [\[CrossRef\]](#)
- Qartah, M.A. Ransomware Economics: Analysis of the Global Impact of Ransom Demands. *J. Inf. Secur.* **2020**.
- Klick, J.; Koch, R.; Br, stetter, T. Epidemic? The attack surface of German hospitals during the COVID-19 pandemic. In Proceedings of the 2021 13th International Conference on Cyber Conflict (CyCon), Tallinn, Estonia, 25–28 May 2021; pp. 73–94.
- Alraizza, A.; Algarni, A. Ransomware detection using machine learning: A survey. *Big Data Cogn. Comput.* **2023**, *7*, 143. [\[CrossRef\]](#)
- Kapoor, A.; Gupta, A.; Gupta, R.; Tanwar, S.; Sharma, G.; Davidson, I.E. Ransomware detection, avoidance, and mitigation scheme: A review and future directions. *Sustainability* **2021**, *14*, 8. [\[CrossRef\]](#)
- Alzubaidi, L.; Bai, J.; Al-Sabaawi, A.; Santamaría, J.; Albahri, A.S.; Al-dabbagh, B.S.N.; Fadhel, M.A.; Manoufali, M.; Zhang, J.; Al-Timemy, A.H.; et al. A survey on deep learning tools dealing with data scarcity: Definitions, challenges, solutions, tips, and applications. *J. Big Data* **2023**, *10*, 46. [\[CrossRef\]](#)
- Shen, L.; Sun, Y.; Yu, Z.; Ding, L.; Tian, X.; Tao, D. On efficient training of large-scale deep learning models: A literature review. *arXiv* **2023**, arXiv:2304.03589.
- Inc, S.C.I. Mutation Effect of Babuk Code Leakage: New Ransomware Variants. *SOCRadar* 2023. Available online: <https://socradar.io/mutation-effect-of-babuk-code-leakage-new-ransomware-variants/> (accessed on 27 April 2024).
- What is Signature-based detection? Understanding Antivirus Signature Detection. Available online: <https://riskxchange.co/1006984/what-is-signature-based-malware-detection/> (accessed on 27 April 2024).
- Sophos. What are Signatures and How Does Signature-Based Detection Work? 2020. Available online: <https://home.sophos.com/en-us/security-news/2020/what-is-a-signature> (accessed on 27 April 2024).
- Odi, J.; Hampo, J.; Nigeria, O.; FO, N.; Onwuama, T. Comparative Analysis of Malware Detection Techniques Using Signature, Behaviour and Heuristics. *Int. J. Comput. Sci. Inf. Secur. IJCSIS* **2019**, *17*, 33–50.
- Mills, A.; Legg, P. Investigating anti-evasion malware triggers using automated sandbox reconfiguration techniques. *J. Cybersecur. Priv.* **2020**, *1*, 19–39. [\[CrossRef\]](#)

20. Gómez-Hernández, J.A.; García-Teodoro, P. Lightweight Crypto-Ransomware Detection in Android Based on Reactive Honeyfile Monitoring. *Sensors* **2024**, *24*, 2679. [CrossRef]
21. Dilhara, B.A.S. Classification of Malware using Machine learning and Deep learning Techniques. *Int. J. Comput. Appl.* **2021**, *183*, 12–17. [CrossRef]
22. Do, N.Q.; Selamat, A.; Krejcar, O.; Herrera-Viedma, E.; Fujita, H. Deep Learning for Phishing Detection: Taxonomy, Current Challenges and Future Directions. *IEEE Access* **2022**, *10*, 36429–36463. [CrossRef]
23. Voulkidis, A.; Skias, D.; Tsekeridou, S.; Zahariadis, T. Network Traffic Anomaly Detection via Deep Learning. *Information* **2021**, *12*, 215. [CrossRef]
24. Tobiyama, S.; Yamaguchi, Y.; Shimada, H.; Ikuse, T.; Yagi, T. Malware Detection with Deep Neural Network Using Process Behavior. In Proceedings of the IEEE 40th Annual Computer Software and Applications Conference (COMPSAC), Atlanta, GA, USA, 10–16 June 2016; Volume 2, pp. 577–582.
25. Alqahtani, A.; Sheldon, F.T. A survey of crypto ransomware attack detection methodologies: An evolving outlook. *Sensors* **2022**, *22*, 1837. [CrossRef]
26. Nguyen, D.T.; Lee, S. LightGBM-based Ransomware Detection using API Call Sequences. *Int. J. Adv. Comput. Sci. Appl. IJACSA* **2021**, *12*, 138–146. [CrossRef]
27. Lin, T.L.; Chang, H.Y.; Chiang, Y.Y.; Lin, S.C.; Yang, T.Y.; Zhuang, C.J.; Zhang, B.H. Ransomware Detection by Distinguishing API Call Sequences through LSTM and BERT Models. *Comput. J.* **2024**, *67*, 632–641. [CrossRef]
28. Mikolov, T.; Chen, K.; Corrado, G.; Dean, J. Efficient estimation of word representations in vector space. In Proceedings of the International Conference on Learning Representations (ICLR 2013), Scottsdale, AZ, USA, 2–4 May 2013.
29. de Oliveira, A.S.; Sassi, R.J. Behavioral Malware Detection Using Deep Graph Convolutional Neural Networks. *Authorea Prepr.* **2023**. Available online: <https://www.authorea.com/users/660121/articles/675292-behavioral-malware-detection-using-deep-graph-convolutional-neural-networks> (accessed on 27 April 2024). [CrossRef]
30. Zhang, S.; Tong, H.; Xu, J.; Maciejewski, R. Graph convolutional networks: A comprehensive review. *Comput. Soc. Netw.* **2019**, *6*, 11. [CrossRef] [PubMed]
31. Karanam, S. Ransomware Detection Using Windows API Calls and Machine Learning. Ph.D. Thesis, Virginia Tech, Blacksburg, VA, USA, 2023.
32. Cover, T.; Hart, P. Nearest neighbor pattern classification. *IEEE Trans. Inf. Theory* **1967**, *13*, 21–27. [CrossRef]
33. Breiman, L. Random forests. *Mach. Learn.* **2001**, *45*, 5–32. [CrossRef]
34. Steinwart, I.; Christmann, A. *Support Vector Machines*; Springer Science & Business Media: New York, NY, USA, 2008.
35. Wright, R.E. Logistic Regression. In *Reading and Understanding Multivariate Statistics*; Grimm, L.G., Yarnold, P.R., Eds.; American Psychological Association: Washington, DC, USA, 1995; pp. 217–244.
36. Maniriho, P.; Mahmood, A.N.; Chowdhury, M.J.M. API-MalDetect: Automated malware detection framework for windows based on API calls and deep learning techniques. *J. Netw. Comput. Appl.* **2023**, *218*, 103704. [CrossRef]
37. Catak, F.O.; Yazı, A.F.; Elezaj, O.; Ahmed, J. Deep learning based Sequential model for malware analysis using Windows exe API Calls. *PeerJ Comput. Sci.* **2020**, *6*, e285. [CrossRef] [PubMed]
38. Alibaba Cloud Malware Detection Based on Behaviors. 2024. Available online: <https://tianchi.aliyun.com/competition/entrance/231694/information?lang=en-us> (accessed on 12 July 2024).
39. Almousa, M.; Basavaraju, S.; Anwar, M. Api-based ransomware detection using machine learning-based threat detection models. In Proceedings of the 2021 18th International Conference on Privacy, Security and Trust (PST), Auckland, New Zealand, 12–15 December 2021; pp. 1–7.
40. Security, H. Windows 7 End of Support: What Does It Mean for Your Organization? 2022. Available online: <https://heimdalsecurity.com/blog/windows-7-end-of-support/> (accessed on 11 May 2024).
41. Microsoft Corporation. *Process Monitor v3.61*, 2023. Available online: <https://techcommunity.microsoft.com/t5/sysinternals-blog/sysmon-v13-00-process-monitor-v3-61-and-psexec-v2-21/ba-p/2048379> (accessed on 24 June 2024).
42. Oracle Corporation. *Oracle VM VirtualBox*, 2023. Available online: <https://www.virtualbox.org/> (accessed on 24 June 2024).
43. Russinovich, M.; Solomon, D.; Ionescu, A. *Windows Internals, Part 1: Covering Windows Server 2008 R2 and Windows 7*; Microsoft Press: Redmond, WA, USA, 2009.
44. Aurangzeb, S.; Aleem, M.; Iqbal, M.A.; Islam, M.A. Ransomware: A survey and trends. *J. Inf. Assur. Secur.* **2017**, *6*, 48–58.
45. Check Point Software Technologies. Different Types of Ransomware, 2024. Available online: <https://www.checkpoint.com/cyber-hub/threat-prevention/ransomware/different-types-of-ransomware/> (accessed on 30 July 2024).
46. VirusShare.com. Available online: <https://virusshare.com/> (accessed on 25 June 2024).
47. Gómez-Hernández, J.; Álvarez González, L.; García-Teodoro, P. R-locker: Thwarting ransomware action through a honeyfile-based approach. *Comput. Secur.* **2018**, *73*, 389–398. [CrossRef]
48. Grave, E.; Bojanowski, P.; Gupta, P.; Joulin, A.; Mikolov, T. FastText Word Vectors. 2018. Available online: <https://fasttext.cc/docs/en/crawl-vectors.html> (accessed on 30 July 2024).
49. Devlin, J.; Chang, M.W.; Lee, K.; Toutanova, K. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *arXiv* **2018**, arXiv:1810.04805.
50. Glorot, X.; Bordes, A.; Bengio, Y. Deep Sparse Rectifier Neural Networks. In Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics (AISTATS), Fort Lauderdale, FL, USA, 11–13 April 2011; pp. 315–323.

51. Rumelhart, D.E.; Hinton, G.E.; Williams, R.J. Learning representations by back-propagating errors. *Nature* **1986**, *323*, 533–536. [\[CrossRef\]](#)
52. LeCun, Y.; Bottou, L.; Bengio, Y.; Haffner, P. Gradient-Based Learning Applied to Document Recognition. *Proc. IEEE* **1998**, *86*, 2278–2324. [\[CrossRef\]](#)
53. Hochreiter, S.; Schmidhuber, J. Long short-term memory. *Neural Comput.* **1997**, *9*, 1735–1780. [\[CrossRef\]](#)
54. Microsoft Corporation. *Microsoft Windows 10 Enterprise Edition*; Microsoft Corporation: Redmond, WA, USA, 2015.
55. Chollet, F. *Deep Learning with Python*; Manning Publications Co.: New York, NY, USA, 2018; ISBN 9781617294433.
56. Pedregosa, F.; Varoquaux, G.; Gramfort, A.; Michel, V.; Thirion, B.; Grisel, O.; Blondel, M.; Prettenhofer, P.; Weiss, R.; Dubourg, V.; et al. Scikit-learn: Machine Learning in Python. *J. Mach. Learn. Res.* **2011**, *12*, 2825–2830.
57. Virtanen, P.; Gommers, R.; Oliphant, T.E.; Haberland, M.; Reddy, T.; Cournapeau, D.; Burovski, E.; Peterson, P.; Weckesser, W.; Bright, J.; et al. SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python. *Nat. Methods* **2020**, *17*, 261–272. [\[CrossRef\]](#)
58. Hunter, J.D. Matplotlib: A 2D Graphics Environment. *Comput. Sci. Eng.* **2007**, *9*, 90–95. [\[CrossRef\]](#)
59. Hastie, T.; Tibshirani, R.; Friedman, J. *The Elements of Statistical Learning*; Springer: Cham, Switzerland, 2009.
60. Fawcett, T. An Introduction to ROC Analysis. *Pattern Recognit. Lett.* **2006**, *27*, 861–874. [\[CrossRef\]](#)
61. Powers, D.M.W. Evaluation: From Precision, Recall and F-Factor to ROC, Informedness, Markedness & Correlation. *J. Mach. Learn. Technol.* **2011**, *2*, 37–63.
62. Rey, D.; Neuhaus, M. Wilcoxon-Signed-Rank Test. In *International Encyclopedia of Statistical Science*; Lovric, M., Ed.; Springer: Berlin/Heidelberg, Germany, 2011. [\[CrossRef\]](#)
63. Gulmez, S.; Kakisim, A.G.; Sogukpinar, I. XRan: Explainable deep learning-based ransomware detection using dynamic analysis. *Comput. Secur.* **2024**, *139*, 103703. [\[CrossRef\]](#)
64. Maniath, S.; Ashok, A.; Poornachandran, P.; Sujadevi, V.; Au, P.S.; Jan, S. Deep learning LSTM based ransomware detection. In Proceedings of the 2017 Recent Developments in Control, Automation & Power Engineering (RDCAPE), Noida, India, 26–27 October 2017; pp. 442–446.
65. Masum, M.; Faruk, M.J.H.; Shahriar, H.; Qian, K.; Lo, D.; Adnan, M.I. Ransomware classification and detection with machine learning algorithms. In Proceedings of the 2022 IEEE 12th Annual Computing and Communication Workshop and Conference (CCWC), Virtual, 26–29 January 2022; pp. 316–322.
66. van der Maaten, L.; Hinton, G. Visualizing Data using t-SNE. *J. Mach. Learn. Res.* **2008**, *9*, 2579–2605.

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.